A Pointfree Algebraic Metatheory of Syntax-Based Systems

XXX*
xxx.yyy@zzz.ww
YYY
ZZZ

Abstract

Semantic notions in programming language theory are commonly specified by syntax-based systems, and their metatheory — including congruence of bisimilarity, determinacy of evaluation, confluence of reduction, and type safety — is typically developed syntactically, relying heavily on termwise reasoning. This leads to representation-dependent results and to a systematic duplication of metatheoretical effort across different calculi and syntactic presentations. This paper proposes a pointfree, algebraic approach to the metatheory of syntax-based systems. Rather than reasoning about terms, inference mechanisms, or other syntactic notions — or about their abstract structure — we shift attention to the algebra of semantic predicates induced by syntax and treat such predicates as first-class objects. This algebra is defined abstractly, without reference to any underlying term structure. Term-based rules and manipulations are recast as algebraic operations, while metatheoretical properties are formulated algebraically — typically as quasi-equations — and established once and for all at the algebraic level, independently of any particular syntactic representation. As a first step towards a systematic algebraization of metatheory, we prove a collection of abstract metatheorems, including confluence of reduction, determinacy of evaluation, and congruence of applicative bisimilarity.

CCS Concepts

• **Do Not Use This Code → Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Keywords

Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

ACM Reference Format:
XXX. 2018. A Pointfree Algebraic Metatheory of Syntax-Based Systems. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 22 pages. https://doi.org/XXXXXXX.XXXXXXX

Conference acronym 'XX, Woodstock, NY

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Semantic notions in programming language theory are most often specified by *syntax-based systems*, such as structural and natural operational semantics [58, 81, 82, 99], rewriting systems [7, 10, 55], proof calculi [37, 116], and type [9, 75, 108] and equational theories [16, 118]. Despite their differences, all these systems can be uniformly understood as modelling semantic notions — typing, reduction, evaluation, equivalence, *etc.* — as suitable predicates and relations on syntactic terms defined by means *syntactic and inferential mechanisms*, such as syntax-directed rules or term-based closure operators.

The Problem. The success of syntax-based systems to provide formal definitions of semantic behaviours is well testified by their widespread use. That use, however, comes at a *metatheoretical* cost. Establishing coherence and correctness of semantic relations — such as *congruence of bisimilarity*, *determinism of evaluation*, *type progress and preservation*, and *confluence of reduction* — can rarely be achieved by *structural reasoning* on these relations alone. On the contrary, metatheoretical arguments are typically carried out *syntactically*, largely resting on concrete aspects of terms and their manipulation that has little to do with their semantic behaviours. This makes reasoning inherently *termwise* and, consequently, the resulting metatheory highly sensitive to presentation choices.

Fundamental metatheorems must often be reproved when moving between calculi or between different presentations of the same calculus, even if nothing essential changes. Even the applicability of general metatheoretical results, such as Aczel's confluence theorem [5] or rule format-based congruence theorems for bisimilarity [3, 117] depends sensitively on whether terms are represented as parse trees [85], abstract binding trees [15, 39], or nominal terms [20, 30], as well as on how substitution is defined, the availability of contexts, metavariables, *etc.* This leads to a duplication of work that is difficult to justify in the long run: *how many times should one re-prove congruence of bisimilarity, confluence of reduction, or type progress and preservation?*

Syntax Structure. A natural response to this situation is to abstract away from concrete representations of syntactic objects by objectifying their essential structure. A large body of work — from the *mathematics of syntax* [29, 38, 47, 52, 53, 91] to *mathematical operational semantics* [15, 39, 117], going through *categorical rewriting* [17, 46, 71, 72, 100, 106] — provides robust and reusable abstractions of syntactic structure, allowing semantic definitions and proofs to be formulated independently of particular syntactic representation of terms, rules, and the like.

Replacing termwise reasoning with reasoning about abstract structures considerably reduces representation-dependence, but does not truly eliminate it; rather, it transforms it into a form of *structure-dependence*. Much as syntactically conducted metatheory

involves reasoning about particular representations of syntactic objects, structural approaches rest on properties of the chosen abstractions themselves, including features that are conceptually peripheral to the metatheoretical phenomena under investigation. A telling illustration is provided by the various, yet equally valid, structures used to model substitution in the presence of variable binding [29, 33, 51, 78, 91]: metatheoretical results that are conceptually orthogonal to this choice — such as congruence of bisimilarity — are typically established only relative to, and hence depend on, one such structure.

1.1 Syntax Relation Algebras

In this paper, continuing [35], we tackle these issues by pursuing a different, complementary path. Rather than objectifying the abstract structure of syntax, we *algebrize* the structure of its 'syntactically-defined' relations, thereby making the latter — and *not* the terms on which they act or the inferential mechanisms through which they are generated — first-class objects.

Our starting point is the twofold observation, exploited in [35], that: *(i)* when defining semantics using syntax-based systems, we are concerned only with certain representation-invariant relations on syntactic objects — *syntax relations* — such as relations invariant for names of bound variable,[1] that naturally carry a rich algebraic structure; *(ii)* syntactic structure, whether abstract or concretely imposed, naturally induces a small set of *operations* on the collection of its syntax relations, which smoothly extends the algebra of these relations. These operations distil the essence of how relations are constructed by structural inspection and manipulation of syntactic terms; inference rules and other syntax-based mechanisms, accordingly, provide concrete realisations of these operations on specific syntactic presentations. Most importantly, the basic structural principles of syntactic reasoning — such as structural induction and compatibility of substitution with term structure — naturally give rise to *algebraic laws* governing these operations, laws that make no reference to any underlying syntactic structure. This makes it possible to abstract away from syntax, retaining an *abstract algebra of syntax relations* defined solely by its operations and laws, and invariant across all syntactic representations.

Prompted by that, the conclusion of [35] advocates a *pointfree* view on syntax-based systems, close in spirit to pointfree topology, and outlines a broad research programme in this direction. Just as pointfree topology studies spaces through the algebra of their open sets, without reference to a predefined underlying set of points, the aim is to study syntax-based systems through the algebra of their syntax relations, defined solely by their operations and equational laws, and without commitment to any specific (abstract or concrete) representation of terms or other syntactic structure.

1.1.1 Contribution, I: Algebraization. In this work, we realise a central and foundational part of this research programme by axiomatising algebras of syntax relations, without reference to any underlying notion of syntax, and developing a substantial body of *pointfree metatheory* on top of such an axiomatisation. We dub the resulting structure *Syntax Relation Algebra* (SRA).

[1] Cf. "the only predicates we ever deal with (when describing properties of syntax) are equivariant ones, in the sense that their validity is invariant under swapping (*i.e.* transposing, or interchanging) names" [96]

SRAs are defined as *quantales* [104] — or, in the finitary case, *action algebras* [64, 101] — extended with a small collection of additional operations whose quasi-equational laws axiomatise the action of syntax on relations. For example, rather than defining principles of structural recursion and induction [18] directly on terms, SRAs include algebraic operations whose defining quasi-equations capture the essential properties of assertions constructed by these principles, but without reference to any underlying term structure. Readers familiar with algebraic logic may view SRAs as algebras of (relationally interpreted) predicates, in the tradition of Tarski[74, 105, 114], enriched with predicate-forming operations induced by syntactic structure; readers familiar with program logic may instead view SRAs as extensions of *Kleene-like algebras* [19, 61, 63] with operators modelling *syntactic actions*.

Even if defined independently of any syntax structure, SRAs are expressive enough to give an algebraic account of many syntactically-defined relations, encompassing those typically use to give semantic to programming languages, such as: many forms of term reduction, type elaboration, big-step and small-step semantics, and the behavioural equivalences and approximations they induce, such as *applicative (bi)similarity* [2], and *contextual* [84] and *CIU* [77] equivalence and approximation.

Furthermore, when viewed with the lens of SRAs, nontrivial syntactic notions — *e.g.* ones of linear occurrence or evaluation context and position — which are typically defined in *ad hoc* and representation-based fashion, are recast as general algebraic constructions on syntax assertions, such as the ones of a *derivative* [86] or *orthogonal decomposition* of a SRA operation.

1.1.2 Contribution, II: Pointfree Metatheory. The real payoff of SRAs, however, lies in the metatheory of syntax-based systems. Contrary to what intuition suggests, many metatheoretical properties of syntax-based systems may be expressed without referring to their underlying terms, taking instead the form of structural properties of syntax relations. These properties typically express that *local coherence* conditions propagates to *global correctness* properties. While these global properties capture the true metatheorems one is interested in — bisimilarity is a congruence, reduction is confluent, *etc.* — local coherence conditions state that *primitive*, or *elementary*, relations, such as ground reductions and equational axioms, are coherent with (operations akin to) syntax structure. On concrete systems, there are plenty of — usually each other *unrelated* — examples of these local conditions, from linearity and orthogonality in rewriting [10, 55] to rule formats in structural operational semantics [3].

Remarkably, the introduction of SRAs not only reveals that these conditions can be uniformly viewed as distinct 'syntactic implementations' of the same elementary laws — such as *Gentzen Inversion* and *Conservation Principle* [36, 48][2] — but also that many and apparently unrelated meta-results follow from essentially the same collection of such laws. As a beautiful example of that, we shall prove that whenever a ground reduction satisfies both the aforementioned inversion and conservation principles, then: *(i)* its

[2] Notice that despite these laws play a crucial, foundational roles in program semantics, type theory, and proof theory — Harper has often referred to them as fundamental principles of language design [48] — they typically lack a formal, syntax-independent formulation.

parallel reduction is confluent; *(ii)* a large families of big-step operational semantics over it (including traditional call-by-name and call-by-value ones [98]) are deterministic; *(iii)* the notion applicative bisimilarity built upon the latter is a congruence.

Notice that these statements make no reference to any syntactic structure or to any other underlying term language: they are abstract SRA statements. As a consequence, once such a statement has been proved — something done *once and for all* — it applies to *any* syntactic structure whose syntax relations form a SRA. And given any syntax-based system built on top of such a syntactic structure, if we want to prove that metatheorems such as confluence reduction or congruence of bisimilarity hold on that system, we do not have to proceed from scratch, applying the same syntactic techniques (*Howe's method* [54, 94], *Tait and Martin-Löf techniques* [8], *etc.*); nor we have to rest on a mosaic of general syntactic theorems, each requiring the verification of diverse representation-dependent conditions in order to be applied. It is sufficient to show that the *primitive* syntax relations of the system satisfy *one and the same* collection of mild coherence conditions: from those, a large body of diverse metatheoretical properties automatically follow.

This is an important point to stress. SRA are not merely a convenient algebraic language for carrying out syntactic reasoning. Their purpose is not to replace termwise or structure-wise proofs with algebraic ones, but to *eliminate the need for such proofs altogether*. By proving metatheorems directly at the level of SRAs, in complete abstraction from any underlying syntax, SRAs make these results immediately applicable to every syntax structure whose relations carry the appropriate algebraic structure.

1.1.3 Contribution, III: Generality and Applicability. At this point, readers may wonder how many syntactic structures do carry a SRA structure, how difficult is to establish that, what kind of metatheorems can we ensure through 'local coherence conditions', and how difficult is to verify those conditions. We answer these questions.

How many syntactic structures do carry a SRAstructure? And how difficult is to endow given structure with a SRA? All the main syntactic structures used in program semantics can be easily endowed with a natural SRA structure. These encompass *free* and many *nonfree* (*e.g.* free structures quotiented by α-renaming) term algebras over a large class of signatures, including: *algebraic* [38], *binding* [5, 62], *nominal* [91], and *monoidal signatures* [57, 107], as well as their multi-sorted and typed refinements. Although endowing a specific language with a SRA structure is typically easy, once can associates each of the aforementioned signature with natural SRAs on their terms (we shall give some examples in the paper, leaving other to the appendix); as a consequence, once users specify their languages through signatures, they will have a corresponding SRA on such languages, *for free*. Finally, under this perspective, we can read [35] as defining a canonical SRA on any suitable initial algebra (possibly extended with substitution structure) on a topos.

What kind of metatheorems can we ensure through 'local coherence conditions'? In this paper, we will prove metatheorems akin to confluence of reduction, determinacy of big-step semantics, and congruence of applicative bisimilarity. Other results we have proved but that we omit due to space constraints include further confluence results (*e.g.* via critical pair-like conditions [86]), equivalence between small-step and big-step semantics, congruence of Kleene and CIU equivalence. Another result we will hint but leave its full analysis to future work is *type safety*.

How difficult is to verify 'local coherence conditions'? This is a typically easy task, mostly done by syntax inspection. Concrete examples of syntax-based systems satisfying the local conditions our main metatheorems build upon include: many λ-calculi [8, 9], such as the untyped call-by-name and call-by-value λ-calculus, and their extensions with additional operators as well as simple, recursive, polymorphic, and depedent types [89, 108]; object calculi, such as the ς-calculus [1] and Featherweight Java [56]; and reduction calculi on string diagrams [12–14]. While, due to space constraints, we will able to show only but a few of these examples, readers should be able to apply our results to the remaining calculi mentioned.

1.1.4 ***Contribution List.*** *(i)* We introduce and axiomatise SRAs as quantales with operators. We witness the 'naturality' of our axiomatisation by showing how the latter derives from transporting canonical categorical structures for syntax — monoidal closed structure, initial algebras, *etc.* — along the embedding of quantales into their many-objectifications. *(ii)* We algebraically define various 'syntactic' notions within SRAs, including ones of an *open* and *closed* expression, *introduction* and *elimination form*, *program*, and *value*. We use those to systematically algebrize general accounts of operational semantics [75]. In doing so, we notice how syntactic notions often regarded as minor, such as the ones of an open and closed expression, once algebrized in the style of SRAs reveal a rich structure, notably a modal and cohesive [68] one, in this case. *(iii)* We use SRA s to prove three fundamental metatheorems in rewriting, operational semantics, and program equivalence, at a high level of generality. Remarkably, we discover that these three metatheorems — *confluence of orthogonal reduction*, *determinism of program evaluation*, and *congruence of applicative bisimilarity* — follow from essentially the same hypotheses — *Gentzen's Inversion and Conservation Principles* — which in turn express a deep relationship between syntax structure and its computational content.

1.2 A Walkthrough Example

Before diving into technical developments, we give an informal introduction to some of the key features of SRAs by commenting one of the metatheorems we shall prove. Here is the (formal statement of the) theorem:

$$a^\circ \,;\, a \leq \Delta \;\&\; a \leq \langle \overline{\Delta} \rangle \,;\, a \implies a^{\Downarrow\circ} \,;\, a^\Downarrow \leq \overline{\Delta} \qquad (\dagger)$$

Informally, (†) states that if a ground reduction a — such as the β-rule — is deterministic ($a^\circ \,;\, a \leq \Delta$) and satisfies Gentzen's Inversion Principle ($a \leq \langle \overline{\Delta} \rangle \,;\, a$), then the big-step semantics it induces ($a^\Downarrow$), evaluates programs to at most one value ($a^{\Downarrow\circ} \,;\, a^\Downarrow \leq \overline{\Delta}$). Let us comment on that:

1.2.1 Syntax Relations. The first observation to make is that we are dealing with relations viewed as *abstract algebraic entities*, and not as predicates acting on terms. Being algebraic entities, relations are synthetically built from 'primitive' relations using various operations, which are conveniently classified into two families.

Logical Operations. Operations such as $\cdot^\circ$, $\cdot_1 \,;\, \cdot_2$, and (the constant) Δ are examples of *logical operations*. These describe, on an

algebraic level, the logical structure of relations, *viz.* the one that is completely independent of (the internal structure of) the related objects [114]. For instance, the aforementioned operations correspond to relation converse, composition, and identity (or diagonal), respectively. These examples also reveal that we are taking a 'binary' view on relations, rather than an n-ary one. Even binary-like relations are enough for our purposes, this is not a real limitation (see Section 2).

Syntactic Operations. The second class of operations, the one characteristic of SRAs, encompasses *syntactic operations*. These operations encapsulate how core syntactic manipulations can be used to define relations. Accordingly, they describe the syntactic structure of relations, the one reflecting the internal (syntactic) structure of the objects on which relations act. At its most fundamental level, syntactic terms are recursively built from variables using operators. Variables, moreover, act as placeholders, and thus they can be substituted for terms. All of that is captured by three operations:

- A constant operations Δ_η, which morally relates two terms if they both are a variable, and the same one.
- A unary operation $\widetilde{\cdot}$ with the following intended meaning: $\widetilde{a}$ relates two terms if they are built using the same outermost operator and their immediate sub-terms are pairwise a-related. SRAs 'force' this intended interpretation *operationally*, through axioms expressing how syntactic operations interact with logical ones — for instance, we require $\widetilde{a\,;b} = \widetilde{a}\,;\widetilde{b}$ — and with each other. Crucially, these axioms constrain the possible interpretations of (otherwise arbitrary) relations to relations on syntactic objects. In particular, by identifying the collection of objects on which relation acts with the diagonal relation Δ, we can recover the (possibly non-algebraic) structure of terms from the (algebraic) structure of their relations.[3] For example, by imposing the equality $\Delta = \widehat{\Delta}$, where we write $\widehat{\cdot}$ for $\Delta_\eta \vee \widetilde{\cdot}$,[4] we restrict terms to those that are either variables or compound terms made of operators and operands (cf. *regularity* of abstract syntax [79]). To make terms recursively generated in that way, it is sufficient to require that Δ is actually the least solution to the equation $x = \widehat{x}$. In this way, we obtain the law [67] $a \leq \widehat{a} \implies \Delta \leq a$ which express *structural induction*.
- A binary operation $\cdot_1[\cdot_2]$ with the following intended meaning: two terms are related by $a[b]$ if the terms are obtained from a-related terms by applying pointwise b-related substitutions on them. The usual equational, monoid-like, theory of substitution is forced precisely by requiring $\cdot_1[\cdot_2]$ to form a monoid on relations, with Δ_η acting as unit (*e.g.* $\Delta_\eta[a] = a = a[\Delta_\eta]$). More remarkably, we can ensure compatibility of substitution with term forming operators through the lax distributivity law $\widetilde{a}[b] \leq \widetilde{a[b]}$. Finally, the possibly non-structural recursive character of substitution corresponds to the existence of the b-parametric family right adjoints $\cdot[b] \dashv b \gg \cdot$.

Using the just described basic operations and the algebraic machinery, we can account for even finer syntactic descriptions. For example, in big-step semantics one splits term constructs into *introductory forms* and *eliminatory forms*, making evaluation reduce programs until an introductory form is reached. Algebraically, this amount to give an *orthogonal decomposition* of $\widetilde{\cdot}$. as $\overline{\cdot} \vee \langle \cdot \rangle$, where $\overline{\cdot}$ and $\langle \cdot \rangle$ restrict the behaviour of $\widetilde{\cdot}$ to introductory forms and eliminatory forms, respectively.

These operations also clarify the role and structural characters of inferential mechanisms, such as rewriting and syntax-directed rules. What rules do, in fact, is precisely to define relations through composition of these operations. For example, rule-based definitions of big-step semantics over a can be algebraically understood as a presentation of $a^{\Downarrow}$, which is in turn the least solution to the algebraic equation $x = \overline{\Delta} \vee \langle x \rangle\,;a\,;x$. The structural character of syntax-based definitions lies precisely in that: they realise algebraic constructions on relations, and these constructions reflect the structure of syntax.

1.2.2 Coherence Conditions. Let us turn to (†). The conclusion of the implication, $a^{\Downarrow\circ}\,;a^{\Downarrow} \leq \overline{\Delta}$, expresses the metatheoretical property of interest, namely that the big-step semantics $a^{\Downarrow}$ is deterministic and evaluates terms to introductory forms. (This may look cryptic at the moment, but once acquainted basic familiarity with algebras of relations it will become crystal clear.) The antecedent, instead, expresses two local conditions that ensure this property. The first is that the ground reduction must be, obviously, itself deterministic. The second, instead, expresses a fundamental property of how computation emerges from syntax structure, namely the already mentioned *Gentzen's Inversion Principle*. This principle asserts that computation arises when an eliminatory form has an introductory form as major argument (a bell should ring for readers familiar with Proposition-as-Types). Notice that these conditions are indeed *local*, as they express properties of a and, indirectly, of the form of syntax. Notice also that their concrete instances are typically easy to check.

2 Preliminaries

We assume familiarity with basic category theory [73] and order theory [24]. In particular, we give the order-theoretic notions of (and standard notations for) poset, meet, join, (complete) lattice, and monotone functions for granted, as well as basic category-theoretic ones, including some entry level vocabulary of enriched category theory [60]. Such categorical notions will be used to show how certain order-theoretic axiomatisation are actually instances of canonical constructions. All reasoning, however, is performed by means of traditional algebraic calculations.

Given monotone functions $F : (A, \leq_A) \rightleftarrows (B, \leq_B) : G$ between posets, we say that F is *left adjoint* to G (and G is *right adjoint* to G), notation $F \dashv G$, if $F(x) \leq_B y \iff x \leq_A G(y)$. We also say that a function F is *join-preserving* (resp. *meet-preserving*) if $F(\bigvee x_i) = \bigvee_i F(x_i)$ (resp. $F(\bigwedge x_i) = \bigwedge_i F(x_i)$). These notions

[3] It is standard in mathematics to infer the non-algebraic structure of an object from the algebra of its (in our case relational) transformations. This passage from [25] nicely expresses that in the case of geometry: "Everywhere in mathematics, we find geometric objects M (like manifolds) that are too complicated for us to understand. One way to make progress is to introduce a vector space V of functions on M. The space V may be infinite-dimensional, but linear algebra is such a powerful tool that we can still say more about the function space V than about the original geometric space." Following this analogy, we may say that SRAs give the linear algebra of the vector space of relational transformations on syntactic objects. (It is an additional pleasant coincidence that there is a strong relationship between linear and relation algebras [105].)

[4] $\vee$ is the usual join operation, whereas $\leq$ is its corresponding ordering.

are related by the (posetal version of) the *Adjoint Functor Theorem* [24, 73]: a function $F : A \to A$ is join-preserving (resp. *meet-preserving*) if and only if it has a right (resp. left) adjoint. This theorem is particularly useful because it allows us to express continuity conditions without resting on infinitary operations simply by requiring the the existence of suitable adjoints.

Finally, we recall that, by Knaster-Tarski Theorem [24], any monotone function $F : A \to A$ on a complete lattice has both *least* and *greatest* fixed points, denoted by μF (or $\mu x.F(x)$) and νF (or $\nu x.F(x)$), respectively. In particular, we can express that an element m is a lest fixed point of F using the fixed point law $F(m) = m$ and fixed point induction principle $F(x) \leq x \implies m \leq x$ (and similarly for greatest fixed points).

2.1 Algebras of Relations

To introduce SRA s, algebras of relations over syntactic objects, we first introduce algebras of relations over *unstructured* entities. Algebraic logic [45] offers various algebras of n-ary relations, the most notorious ones perhaps being *cylindric* [49] and *polyadic algebras* [44]. For our purposes, however, it is enough[5] — and more convenient — to take a 'binary' view on relations, as embodied by classic relation algebras [74, 102, 114] and its fragments (action algebras [101], Kleene algebras [19, 61, 63], *etc.*). Among those algebras, we choose *quantales* [104], which we introduce, in order to give categorical justification to the axioms defining SRAs, through *quantaloids* [92, 112]. Readers not interested in such a justrification, can stick with the ordinary order-theoretic presentation.

We write arrow composition in a category in diagrammatic order (and notation), and denote the identity arrow by Δ (for diagonal).

Definition 2.1. *1. An* involutive quantaloid *is a* dagger sup-enriched *category $\mathcal{A}$. That is: (i) hom-sets $\mathcal{A}(A, B)$ form a sup-semilattice with composition distributing over joins in both arguments i.e. $(\bigvee_i a_i)\ ;\ b = \bigvee_i a_i\ ;\ b$ and $a\ ;\ (\bigvee_i b_i) = \bigvee_i a\ ;\ b_i$; (ii) there is an involution $\cdot^\circ : \mathcal{A}(A, B) \to \mathcal{A}(B, A)$, that is: $a^{\circ\circ} = a$, $(a\ ;\ b)^\circ = b^\circ\ ;\ a^\circ$, $\Delta^\circ = \Delta$, and $(\bigvee a_i)^\circ = \bigvee_i a_i^\circ$.*

2. A morphism *between quantaloids $\mathcal{A}, \mathcal{A}'$ is a functor $F : \mathcal{A} \to \mathcal{A}'$ that preserves joins and involutions.*

3. An oplax morphism *between quantaloids $\mathcal{A}, \mathcal{A}'$ is an oplax functor $F : \mathcal{A} \to \mathcal{A}'$ (i.e. $a \leq b \implies F(a) \leq F(b), F(a;b) \leq F(a);F(b)$, $F(\Delta) \leq \Delta$) that preserves involutions oplaxly. A* lax morphism *is defined similarly by taking lax functors and requiring lax preservation of involution.*

4. An oplax functor that is oplax on identities but otherwise exact is said to be weakly unital.

5. An involutive quantale *is a one-object quantaloid. Notions of exact, lax, oplax, and weakly unital quantale morphisms are defined accordingly.*

6. Two functions $F : \mathcal{A}^n \to \mathcal{A}, G : \mathcal{A}^m \to \mathcal{A}$ on a quantale $\mathcal{A}$ are orthogonal *if $F(x_1, \ldots, x_n)\ ;\ G(y_1, \ldots, y_m) \leq \bot$.*

We refer to involutive quantales simply as 'quantales'. Moreover, since we think about the elements of quantale as abstract relations, we employ a 'relational vernacular'. We refer to elements of a quantale as *relations*, to Δ as the *identity*, or *diagonal*, relation, and to the operations ; and $-^\circ$ as *(relation) composition* and *converse*, respectively. By Knaster-Tarski theorem, the monotone a-parametric function $\Delta \vee a\ ;\ \cdot$ has a least fixed point, denoted by a^*. We refer to the corresponding operation $\cdot^*$ as *iteration* Finally, by the posetal version of adjoint functor theorem, any quantale $\mathcal{A}$ comes with *right adjoints* $\to, \leftarrow$ of (the unary $\mathcal{A}$-parametric functions induced by) ;. We sometimes referred to $\to$ and $\leftarrow$ as *right* and *left implications*, respectively [101].

Example 2.2. 1. The prototypical example of a quantale is given by set-theoretic endorelations. For any set W, the powerset $\wp(W \times W)$ carries the structure of a quantale. In particular, we have [101]: $\phi \to \psi = \{(w, v) \mid \forall u \in W.\ (u, w) \in \phi \Rightarrow (u, v) \in \psi\}$ and $\psi \leftarrow \phi = \{(w, v) \mid \forall u \in W.\ (v, u) \in \phi \Rightarrow (w, u) \in \psi\}$.

2. Given a set I and a set W, we consider *dependent sets*, namely families $A \in \prod_{i \in \wp_{\text{fin}}(I)} \wp(W)$ of sets indexed by finite subsets of I. As usual, we sometimes write a dependent set A as the sequence $(A(i) \mid i \in \wp_{\text{fin}}(I))$. A *dependent relation* ϕ between dependent sets A and B is an element in $\prod_i \wp(A(i), B(i))$ *closed under renaming*: for any function (renaming) $f : i \to j$, if $(\mathtt{t}, \mathtt{s}) \in \phi(i)$, then $(\mathtt{t}, \mathtt{s}) \in \phi(j)$. With these definitions, the quantale structure of $\wp(A(i), A(i))$ extends pointwise to $\prod_i \wp(A(i), A(i))$, endowing the collection of dependent endorelations over a dependent set with a quantale structure, too [92].

We now introduce a basic construction to turn a quantale $\mathcal{A}$ into a many-objects (small) quantaloid $\mathcal{A}^\dagger$ in such a way that there is a full and faithful emebedding $\mathsf{q} : \mathcal{A} \hookrightarrow \mathcal{A}^\dagger$ of $\mathcal{A}$ into $\mathcal{A}^\dagger$. In this way, we can endow $\mathcal{A}$ with algebraic structure by endowing $\mathcal{A}^\dagger$ with (typically well-established) categorical structure and transporting this structure along the embedding $\mathsf{q} : \mathcal{A} \hookrightarrow \mathcal{A}^\dagger$. This construction will allow us to give more compact and conceptually clear definitions of SRA.

Definition 2.3. *Fixed a quantale $\mathcal{A}$, we say that a relation $a \in \mathcal{A}$ is: (i)* reflexive *if $\Delta \leq a$; (ii)* co-reflexive *if $a \leq \Delta$. (iii)* symmetric *if $a^\circ \leq a$; (iv)* transitive *if $a\ ;\ a \leq a$; (v)* co-transitive *if $a \leq a\ ;\ a$; (vi) a* preorder *if it is reflexive and transitive (vii) an* equivalence *if it is a symmetric preorder. (viii) a* co-equivalence *if it is symmetric, co-transitive, and co-reflexive .*

It is easy to see that $a \vee \Delta$, a^*, and $(a \vee a^\circ)^*$ are the least reflexive relation, preorder, and equivalence extending a, respectively. Co-equivalences correspond to unary relations, and thus as collections of objects, the identity relation acting as the collection of all objects. Indeed, given two co-equivalence relations p, q, we see that $p;q$ gives the meet of p and q. In this way, any property P on elements can be regarded as a co-equivalence p (on $\wp(W \times W)$, for instance, any $P \in \wp(W)$ defines a co-equivalence $p \triangleq \{(x, y) \mid x = y\ \&\ P(x)\}$). Proving that any object has the property P then amounts to showing $\Delta \leq p$. Furthermore, we may read a relation of the form $p\ ;\ a\ ;\ q$ as the relation a whose domain and codomain have been restricted to elements satisfying p and q, respectively.

Now it comes the announced construction: any quantale gives a many-objects (small) quantaloid by splitting its co-equivalences into objects.

[5] One can argue, after [115], that algebras of binary relations are "[. . .] strong enough to be called a truly first-order (as opposed to propositional) algebraic logic" [6]. Furthermore, it is well-known [32] that the addition of duplication operators enhance the expressiveness of algebras of binary relations to relations of arbitrary rank.

Proposition 2.4. *Given a quantale $\mathcal{A}$, viewed as a quantaloid on a single object $*$, we construct the quantaloid $\mathcal{A}^\dagger$ — the* splitting of $\mathcal{A}$ — *whose objects are co-equivalences $p \in \mathcal{A}(*,*)$ and whose morphisms $a : p \to q$ are relations $a \in \mathcal{A}(*,*)$ such that $a = p\,;a\,;q$. The identity morphism is given by diagonal relation Δ, whereas the composition of $a : p \to q$ and $b : q \to r$ is the relation $a\,;b$. Then, we have a full and faithfull embedding $\mathrm{q} : \mathcal{A} \hookrightarrow \mathcal{A}^\dagger$ that maps $*$ to Δ and $a \in \mathcal{A}(*,*)$ to $a : \Delta \to \Delta$. Notice also that any hom-set $\mathcal{A}^\dagger(p,q)$ can be embedded into $\mathcal{A}^\dagger(\Delta,\Delta)$.*

3 Syntax Relation Algebras: Basic Definitions

Quantales (and similar structures) come with the fundamental limitation of being agnostic to the structure of the entities on which relations act. SRAs overcome this limitation by extending quantales with operations that axiomatise the action of syntax on relations.

The essential structure of syntax, the one we have informally described in Section 1.2, is usually formalised using categorical constructions akin to *initial algebras* [38, 66], *substitution monoids* [28, 29], *etc.* Sometimes, however, one encounters syntactic structures that do not give rise to such categorical constructions, or they do so but in nontrivial ways. SRAs overcome these problems by imposing the aforementioned structures not on terms, but on relations between them. SRAs, in fact, arise by imposing simple categorical structures on the many-objectification of quantales; notably, an endofunctor Σ giving an initial Σ-algebra structure to the identity relation, and a closed monoidal structure for substitution.[6] Once we transport this structure to the original quantale, we obtain a small collection of operations subject to natural quasi-equational laws. We define SRAs precisely as these resulting structures.

We remark that the just outlined categorical structure has only an explanatory function: once we have a syntactic structure with a corresponding quantale of syntax relations on it, it is enough to define the 'concrete' SRA operations and verify the axioms hold, without involvement of any categorical notion or construction. Sticking with traditional algebra yields many advantages, both in terms of metatheoretical reasoning and applicability. On the one hand, reasoning collapses to traditional algebraic calculations; on the other hand, remarkably many concrete syntactic structures can be readily endowed with a SRA structure.

Definition 3.1. *A SRA is a quantale $\mathcal{A}$ extended with a co-equivalence Δ_η, a weakly unital morphism $\widetilde{\cdot}$, and an oplax morphism $\cdot_1[\,\cdot_2]$ that strictly preserves converse in both arguments and preserves joins in the first argument, all subject to the following conditions, where we write $\widehat{\cdot}$ for the derived operation $\Delta_\eta \vee \widetilde{\cdot}$:*

1. *Δ_η and $\widetilde{\cdot}$ are* orthogonal.
2. *The identity relation Δ is the* least fixed point *of $\widehat{\cdot}$, that is: $\widehat{\Delta} = \Delta$ and $\widehat{a} \leq a \implies \Delta \leq a$.*
3. *$(\mathcal{A}, \Delta_\eta, \cdot_1[\,\cdot_2])$ forms a monoid.*
4. *$\cdot\,[b]$ distributes over $\widetilde{\cdot}$. That is: $\widetilde{a}[b] \leq \widetilde{a[b]}$.*

In any SRA, the a-parametric equation $x = \widehat{x}\,;a$ has a least solution — denoted by $a^{\mathbf{H}}$ — by Knaster-Tarski theorem (notice that such a least solution is actually the *unique* solution to that equation). As we shall see, the resulting family of solutions captures definition by *structural recursion* and corresponds to *algebra initiality*. We thus explicitly consider a unary operation $\cdot^{\mathbf{H}}$ subject to the laws $a^{\mathbf{H}} = \widehat{a^{\mathbf{H}}}\,;a$ and $\widehat{x}\,;a \leq x \implies a^{\mathbf{H}} \leq x$.[7]

3.0.1 Discussion. Let us comment on Definition 3.1. We have already given some explanations of the informal meaning of defining operations of SRA (and more will be said when analysing concrete examples). But what about their mathematical status? Do they arise as instances of 'natural' constructions? The answer is in the affirmative.

Initiality. Let us consider the splitting $\mathrm{q} : \mathcal{A} \hookrightarrow \mathcal{A}^\dagger$ of a quantale $\mathcal{A}$. Following the initial algebra approach to syntax [38], we model variables and operators — their action on relations — using a monotone endo-functor $F = \Delta_\eta + \Sigma : \mathcal{A}^\dagger \to \mathcal{A}^\dagger$, where Δ_η is an object (*i.e.* co-equivalence) standing for variables and Σ is weakly unital. We see that the object Δ_η gives an arrow $\Delta_\eta \in \mathcal{A}^\dagger(\Delta,\Delta)$, and thus, transporting along q, a relation in $\mathcal{A}$. Furthermore, if Δ has a Σ-algebra structure $s : \Sigma\Delta \to \Delta$, then Σ gives an operation $\widetilde{\cdot}$ on $\mathcal{A}^\dagger(\Delta,\Delta)$ defined by $\widetilde{a} \triangleq s^\circ\,;\Sigma a\,;s$. Finally, if we require initiality of Δ for F, then for any $a : Fp \to p$, we obtain the unique initial extension $(\!|a|\!) : \Delta \to p$. This induces a unary operation $\cdot^{\mathbf{H}}$ on $\mathcal{A}^\dagger(\Delta,\Delta)$ simply by turning $a : \Delta \to \Delta$ into $i\,;a : F\Delta \to \Delta$, where $i : F\Delta \to \Delta$ is the F-algebra structure, and defining $a^{\mathbf{H}}$ as $(\!|i\,;a|\!)$. This reveals that 'the-non-substitution-fragment' of a SRA emerges by transporting initial algebra semantics along $\mathrm{q} : \mathcal{A} \hookrightarrow \mathcal{A}^\dagger$ (verifying that the algebraic laws of SRAs follow from the universal properties of initial algebra is straightforward).

Substitution. Substitution is obtained in a smooth by imposing a closed monoidal structure on $\mathcal{A}^\dagger$ and then transporting that structure along q. We endow $\mathcal{A}^\dagger$ with an oplax bifunctor that strictly preserves converse $\otimes : \mathcal{A}^\dagger \times \mathcal{A}^\dagger$ such that: *(i)* $(\mathcal{A}^\dagger, \Delta_\eta, \otimes)$ is a closed monoidal category, with $\otimes\, p \dashv p \multimap \cdot$; *(ii)* there is a right monoidal strength $\Sigma(\,\cdot_1) \otimes (\,\cdot_2) \leq \Sigma(\,\cdot_1 \otimes \,\cdot_2)$. As before, transporting this structure along q we see that monoidal product $\cdot_1 \otimes \cdot_2$ gives rise to the monoid multiplication $\cdot_1[\,\cdot_2]$ and that the right monoidal strength gives distributivity of $\cdot\,[b]$ over $\widetilde{\cdot}$. Finally, the adjunction $\otimes\, b \dashv b \multimap \cdot$ determines the (posetal) adjunction $\cdot\,[b] \dashv b \mathbin{\rightarrowtail} \cdot$ which, by the posetal Adjoint Functor Theorem, gives join-preservation of $\cdot\,[b]$. Notice that the requirement of join-preservation captures the *recursive* — albeit not necessarily *structural recursive* ((see [97] for a nice explanation)) — character of substitution. The existence of right adjoints $\cdot\,[b] \dashv b \mathbin{\rightarrowtail} \cdot$, gives an elegant to capture that very same trait using finitary operations only.

Remark 3.2. In light of the just given comment, we can define a SRA as a one-object quantaloid endowed with the required initial algebra and monoidal closed structure.

[6] Remarkably, this simple construction works also for languages with variable binding, where a similar construction fails when performed on terms, and nontrivial adjustments are needed [28, 29]. This further witness the advantages gained by shifting from terms to relations.

[7] While we build upon quantales, one may prefer to use *finitary* operations only. All our development, except for the definition of (bi)similarity (whose existence has to be imposed by definition), can be carried out with finitary SRAs. These are defined by replacing quantales with *action algebras*, join-preservation of $\cdot\,[b]$ with the existence of right adjoints $\cdot\,[b] \dashv b \mathbin{\rightarrowtail} \cdot$ (something we can safely done, by the Adjoint Functor Theorem), and taking $\cdot^{\mathbf{H}}$ as primitives, together with its corresponding fixed point equation and fixed point induction principle. Notice that the resulting structure is a *quasi-variety* [16, 118]. The vast majority of our proofs have been given within this finitary fragment using quasi-equational logic.

3.1 Examples

Before studying the expressiveness of SRAs, we introduce some familiar examples. Through the rest of this work, we refer to Δ_η, $\widecheck{\cdot}$, $\widehat{\cdot}$, $\cdot^{\mathbf{H}}$, and $\cdot_1[\cdot_2]$ as the *variable co-equivalence* [35], *strict compatible refinement* [35, 69], *compatible refinement* [41, 67], *Howe extension* [54], and *relation substitution* [67], respectively, for reasons that will become clear soon.

3.1.1 First-Order Syntax. Let us fix a first-order signature Σ (*i.e.* a collection of operation symbols $\mathbf{op}$ together with an arity function $|\mathbf{op}| \in \mathbb{N}$) and a disjoint collection $\mathcal{V}$ of variables. We refer to elements in $\wp_{\mathrm{fin}}(\mathcal{V})$ as *contexts* and to maps $\rho : \Gamma \to \Delta$ between contexts Γ, Δ as *renaming* of Γ into Δ.

Definition 3.3. *The dependent set of* terms in context $\mathcal{T}_\Sigma$ *is defined inductively, via the following rules, in which we write* $\Gamma \vdash \mathtt{t}$ *in place of* $\mathtt{t} \in \mathcal{T}_\Sigma(\Gamma)$. *The collection* $\mathcal{T}_\Sigma(\mathcal{V})$ *of all terms is defined as* $\bigcup_\Gamma \mathcal{T}_\Sigma(\Gamma)$.

$$\frac{\mathtt{x} \in \Gamma}{\Gamma \vdash \mathbf{var}\ \mathtt{x}} \qquad \frac{\Gamma \vdash \mathtt{t}_1 \quad \cdots \quad \Gamma \vdash \mathtt{t}_{|\mathbf{op}|} \quad \mathbf{op} \in \Sigma}{\Gamma \vdash \mathbf{op}(\mathtt{t}_1, \ldots, \mathtt{t}_{|\mathbf{op}|})}$$

Notice that the collection variables itself induces a dependent set $\mathcal{V}(\Gamma) \triangleq \Gamma$, and that any context renaming $\rho : \Gamma \to \Delta$ extends to a term renaming $-[\rho] : \mathcal{T}_\Sigma(\Gamma) \to \mathcal{T}_\Sigma(\Delta)$. Finally, rule induction [4] provides the familiar principles of structural induction and recursion on $\mathcal{T}_\Sigma(\mathcal{V})$.

A *substitution* is a map $\sigma : \mathcal{V} \to \mathcal{T}_\Sigma(\mathcal{V})$ with finite support, meaning that there are only finitely many variables $\mathtt{x}$ such that $\sigma(\mathtt{x}) \neq \mathbf{var}\ \mathtt{x}$). Using either rule induction or structural recursion, one can extend the action of substitution to terms in the usual way (see, *e.g.*, [93]), thus obtaining a map $-[\sigma] : \mathcal{T}_\Sigma(\mathcal{V}) \to \mathcal{T}_\Sigma(\mathcal{V})$. Writing id for the identity substitution $id(\mathtt{x}) \triangleq \mathbf{var}\ \mathtt{x}$ and $(\sigma \cdot \tau)(\mathtt{x}) \triangleq \sigma(\mathtt{x})[\tau]$ for the composition of substitutions σ, τ, we obtain the monoid equational theory of substitution: $(\mathbf{var}\ \mathtt{x})[\sigma] = \sigma(\mathtt{x})$, $\mathtt{t}[id] = \mathtt{t}$, and $\mathtt{t}[\sigma])[\tau] = \mathtt{t}[\sigma \cdot \tau]$. Furthermore, we see that substitution is *syntax-preserving*: $\mathbf{op}(\mathtt{t}_1, \ldots, \mathtt{t}_{|\mathbf{op}|})[\sigma] = \mathbf{op}(\mathtt{t}_1[\sigma], \ldots, \mathtt{t}_{|\mathbf{op}|}[\sigma])$.

By Example 2.2, we know that we have a quantale on (first-order) terms in context. Notice how invariance under renaming captures our intuition of invariance of assertions under irrelevant syntactic details. To obtain a SRA, we first observe that the very definition of terms gives a definition of the variable co-equivalence and strict compatible refinement thus (we write $\Gamma \vdash \mathtt{t}\ \phi\ \mathtt{s}$ in place of $(\mathtt{t}, \mathtt{s}) \in \phi(\Gamma)$):

$$\frac{\mathtt{x} \in \Gamma}{\Gamma \vdash \mathbf{var}\ \mathtt{x}\ \Delta_\eta\ \mathbf{var}\ \mathtt{x}} \qquad \frac{\Gamma \vdash \mathtt{t}_1\ \phi\ \mathtt{s}_1 \quad \cdots \quad \Gamma \vdash \mathtt{t}_{|\mathbf{op}|}\ \phi\ \mathtt{s}_{|\mathbf{op}|}}{\Gamma \vdash \mathbf{op}(\mathtt{t}_1, \ldots, \mathtt{t}_{|\mathbf{op}|})\ \widetilde{\phi}\ \mathbf{op}(\mathtt{s}_1, \ldots, \mathtt{s}_{|\mathbf{op}|})}$$

Notice that these definitions are *not* inductive, but they rather describe a single step in the inductive construction of terms, the latter relationally corresponding to Δ. This precisely makes Δ the least fixed point of $\widehat{\cdot}$. The law $\widehat{\phi} \leq \phi \implies \Delta \leq \phi$ thus captures structural induction [67]. In fact, taking ϕ as a co-equivalence, the above quasi-equation states that to prove that any term has the property ϕ (*i.e.* $\Delta \leq \phi$), it is enough to show that any variable satisfies ϕ (*i.e.* $\Delta_\eta \leq \phi$) and that the collection of terms satisfying ϕ is closed under term constructors (*i.e.* $\widetilde{\phi} \leq \phi$).

Let us now move to the Howe extension operation. By instantiating its least fixed point characterisation, we obtain the the following inductive definition:

$$\frac{\Gamma \vdash \mathbf{var}\ \mathtt{x}\ \phi\ \mathtt{t}}{\Gamma \vdash \mathbf{var}\ \mathtt{x}\ \phi^{\mathbf{H}}\ \mathtt{t}} \qquad \frac{\Gamma \vdash \mathtt{t}_1\ \phi^{\mathbf{H}}\ \mathtt{s}_1 \quad \cdots \quad \Gamma \vdash \mathtt{t}_{|\mathbf{op}|}\ \phi^{\mathbf{H}}\ \mathtt{s}_{|\mathbf{op}|} \quad \Gamma \vdash \mathbf{op}(\mathtt{s}_1, \ldots, \mathtt{s}_{|\mathbf{op}|})\ \phi\ \mathtt{s}}{\Gamma \vdash \mathbf{op}(\mathtt{t}_1, \ldots, \mathtt{t}_{|\mathbf{op}|})\ \phi^{\mathbf{H}}\ \mathtt{t}}$$

Readers familiar with *Howe's method* [54], will probably recognise that $a^{\mathbf{H}}$ is precisely the *Howe extension* [41, 54, 90] of the relation ϕ. Readers familiar with rewriting, will probably see that $a^{\mathbf{H}}$ follows the same pattern of *parallel reduction* [113] in the style of Tait and Martin-Löf [8]. Indeed, these constructions are instances of the same general notion, which is, as previously seen, a form of relational initiality (see also [35]).

Finally, the action of substitution naturally extends from terms to relations, giving rise to the relation substitution operation:

$$\Delta \vdash \mathtt{t}[\tau]\ \phi[\psi]\ \mathtt{s}[\sigma] \iff \exists \Gamma.\, \Delta \vdash \mathtt{t}\ \phi\ \mathtt{s}\ \&\ \forall \mathtt{x} \in \Gamma.\, \Delta \vdash \tau(\mathtt{x})\ \psi\ \sigma(\mathtt{x})$$

With this definition, the monoid laws of substitution determine an actual monoid structure $(\mathcal{A}, \Delta_\eta, \cdot_1[\cdot_2])$ on relations. Syntax-preservation of substitution then gives the law $\widetilde{a}[b] \leq \widetilde{a[b]}$, whereas $\Gamma \vdash \mathtt{t}\ (\phi \gg \psi)\ \mathtt{s}$ is defined by

$$\forall \tau, \sigma, \Delta.[(\forall \mathtt{x} \in \Gamma.\, \Delta \vdash \tau(\mathtt{x})\ \phi\ \sigma(\mathtt{x})) \implies \Delta \vdash \mathtt{t}[\tau]\ \psi\ \mathtt{s}[\sigma]]$$

Remark 3.4. The reader may have recognised the pattern behind logical relations. Indeed, logical relations seem much related with solution to the equation $x = x \gg x$. Notice however that, since $\gg$ is antitone in the first argument, the existence of such a solution is not ensured by standard fixed point arguments. We leave this for future research.

3.1.2 Second-Order Syntax. Second-order syntax [5, 29] enriches first-order syntax with binding structure. Loosely, this means that term constructs can bind variables, and that terms are identified modulo renaming of bound variables. In this example, we model second-order syntax through *binding signatures* [5], an extension of first-order signatures where the arity of an operation symbol is an element of $\mathbb{N}^*$. The intended meaning of an arity $|\mathbf{op}| = (k_1, \ldots, k_n)$ (notation (notation $\mathbf{op} : |\mathbf{op}|$) is that operation $\mathbf{op}$ takes n arguments and binds k_i variables in the i-th argument. Because we shall deal with renaming of bound variables, it is necessary to have an unlimited supply of variables. Accordingly, from now on, we tacitly assume that $\mathcal{V}$ is countably infinite.

Definition 3.5. *The collection* $\mathcal{T}_\Sigma(\mathcal{V}) \triangleq \bigcup_\Gamma \mathcal{T}_\Sigma(\Gamma)$ *of* raw terms *in context is defined similarly to the first-order case, by the following rules, where the notation* $\Gamma, \mathtt{x}$ *stands for* $\Gamma \cup \{\mathtt{x}\}$, *provided that* $\mathtt{x} \notin \Gamma$, *and each* $\vec{\mathtt{x}}_i$ *has the right length* k_i*:*

$$\frac{\mathtt{x} \in \Gamma}{\Gamma \vdash \mathbf{var}\ \mathtt{x}} \qquad \frac{\Gamma, \vec{\mathtt{x}}_1 \vdash \mathtt{t}_1 \quad \cdots \quad \Gamma, \vec{\mathtt{x}}_n \vdash \mathtt{t}_n \quad |\mathbf{op}| = (k_1, \ldots, k_n)}{\Gamma \vdash \mathbf{op}(\vec{\mathtt{x}}_1.\mathtt{t}_1, \ldots, \vec{\mathtt{x}}_n.\mathtt{t}_n)}$$

As usual, in a raw term of the form $\mathbf{op}(\vec{\mathtt{x}}_1.\mathtt{t}_1, \ldots, \vec{\mathtt{x}}_n.\mathtt{t}_n)$, we say that variables $\vec{\mathtt{x}}_i$ are *binding* in $\mathbf{op}$ and *bound* (if present) in $\mathtt{t}_i$; variables not bound in a term are said to be *free*, whereas a variable not appearing neither in Γ nor in $\mathtt{t}$ is said to be *fresh* for the judgment $\Gamma \vdash \mathtt{t}$

Example 3.6. *(i)* As a running example, we consider the signature of the λ-calculus, made of two operations — $\mathtt{lam}$ and $\mathtt{app}$ — of arity (1) and $(0, 0)$, respectively. We will employ more familiar notation,

writing, *e.g.*, λx.t and ts in place of lam(x.t) and app(t, s), respectively. *(ii)* It is easy to extend this signature to more sophisticated languages, such as calculi with dependent product types and natural numbers [75, 76]. For instance, we may consider the binding signature containing operators Pi : (0, 1) (in more familiar notation, Π(x : t).s, with the first term standing for a type); lam : (0, 1) (λ(x : t).s, with t acting as a type) eq : (0, 0, 0); refl : (0); eqrec : (0, 0); nat : (); z :: (); succ : (0); natrec : (0, 0, 2).

Any binding signature induces a notion of *variable renaming* on raw terms and a corresponding *equivalence under variable-renaming* — so-called α-equivalence. Both these notions are defined by rule induction (the reader can find all details in the lecture notes by Pitts [93]), and give a context-dependent equivalence relation $\sim_\alpha$ on raw terms.

Definition 3.7. *We define the dependent set of* terms *in context* Γ *as the* quotient $\mathcal{T}^\alpha_\Sigma(\Gamma) \triangleq \mathcal{T}_\Sigma(\Gamma)/{\sim_\alpha}$*, and the collection of terms as* $\mathcal{T}^\alpha_\Sigma(\mathcal{V}) \triangleq \bigcup_\Gamma \mathcal{T}^\alpha_\Sigma(\Gamma)$*. As before, dependent relations on dependent sets* $\mathcal{T}^\alpha_\Sigma$ *form a quantale.*

As a notational convention, we do not distinguish between raw terms t and their $\sim_\alpha$-equivalence classes — *terms* — $[\mathtt{t}]_{\sim_\alpha}$. Most definitions on terms are actually given on raw terms, and then tacitly extended to terms by (equally tacitly) observing that these definitions are invariant under $\sim_\alpha$. For example, we define $\mathcal{T}^\alpha_\Sigma(\mathcal{V})$ strict compatible refinement first on raw terms, and then observe that the resulting notion is invariant under $\sim_\alpha$. The definition of Δ_η is given similarly, and it is essentially identical to the one given on first-order terms:

$$\frac{\Gamma, \vec{\mathtt{x}}_1 \vdash \mathtt{t}_1 \mathrel{\phi} \mathtt{s}_1 \quad \cdots \quad \Gamma, \vec{\mathtt{x}}_n \vdash \mathtt{t}_n \mathrel{\phi} \mathtt{s}_n \quad |\mathbf{op}| = (k_1, \ldots, k_n)}{\Gamma \vdash \mathbf{op}(\vec{\mathtt{x}}_1.\mathtt{t}_1, \ldots, \vec{\mathtt{x}}_n.\mathtt{t}_n) \mathrel{\widetilde{\phi}} \mathbf{op}(\vec{\mathtt{x}}_1.\mathtt{s}_1, \ldots, \vec{\mathtt{x}}_n.\mathtt{s}_n)}$$

Moving to substitution, the shift from first-order to second-order syntax requires us to deal with capture-avoidance. A substitution is a map $\sigma : \mathcal{V} \to \mathcal{T}^\alpha_\Sigma(\mathcal{V})$ with finite support, and we would like to extend it to a map $-[\sigma] : \mathcal{T}^\alpha_\Sigma(\mathcal{V}) \to \mathcal{T}^\alpha_\Sigma(\mathcal{V})$ satisfying monoid laws and syntax-preservation (that is one of the reasons why capture-avoidance is needed). Since maps $-[\sigma]$ correspond to maps $-\{\sigma\} : \mathcal{T}_\Sigma(\mathcal{V}) \to \mathcal{T}^\alpha_\Sigma(\mathcal{V})$ that are invariant under $\sim_\alpha$ (*i.e.* $\mathtt{t} \sim_\alpha \mathtt{s} \implies \mathtt{t}\{\sigma\} = \mathtt{s}\{\sigma\}$), it is enough to extend σ to $-\{\sigma\}$.

It is well-known that naive attempts to that by structural recursion on $\mathcal{T}_\Sigma(\mathcal{V})$ fail, so that one has to rest on different techniques. Nominal sets [33] (see below) provide an elegant solution to the problem, offering structural definitions of substitution by α-structural recursion [97]. Working with binding signature, we can define substitution as an inductive relation on raw terms, then showing, by rule induction, that the resulting relation gives a function on $\sim_\alpha$-equivalence classes of raw terms [93]. We could even work with $\sim_\alpha$-equivalence classes of abstract syntax trees and define substitution by natural number recursion, as customary in classic textbooks on the λ-calculus [8, 50, 65], or following the clever, *first-order*, approach by Stoughton [109]. In all these cases, we obtain a definition of capture-avoiding substitution upon which we define relation substitution as in the previous example.

Nominal Syntax. Nominal sets [91] provide a general approach to second-order syntax based on the notions of permutation and finite support. In a nutshell, given a group G, a *nominal set* consists of a set X endowed with a G-action $\cdot : G \times X \to X$ on it, such that all elements of X are finitely supported. When it comes to syntax, one considers the group of *permutations* $\mathrm{Sym}(\mathcal{V})$ of the set of variables $\mathcal{V}$. This immediately makes $\mathcal{V}$ a nominal set, with $\mathrm{Sym}(\mathcal{V})$-action given by function application.

Definition 3.8. *Given a binding signature* Σ *and a set of variables* $\mathcal{V}$*, we define the* $\mathrm{Sym}(\mathcal{V})$*-action on* $\mathcal{T}_\Sigma(\mathcal{V})$ *by:* $\pi \cdot \mathbf{var}\ \mathtt{x} \triangleq \pi(\mathtt{x})$ *and* $\pi \cdot \mathbf{op}(\vec{\mathtt{x}}_1.\mathtt{t}, \ldots, \vec{\mathtt{x}}_n.\mathtt{t}_n) \triangleq \mathbf{op}(\overrightarrow{\pi(\mathtt{x})}_1.(\pi \cdot \mathtt{t}), \ldots, \overrightarrow{\pi(\mathtt{x})}_n.(\pi \cdot \mathtt{t}_n))$*. This way, we obtain a* nominal set *of raw terms.*

The natural notion of a term relation on nominal raw terms is the one of an *equivariant finitely supported relation* [91, 95], which gives the desired SRA structure on $\mathcal{T}_\Sigma(\mathcal{V})$ by the so-called Finite Support Principle (see [91] and Theorem 3.5 in [97]). More interestingly, nominal sets give a natural notion of α-equivalence $\sim_\alpha$, and the quotient set $\mathcal{T}^\alpha_\Sigma(\mathcal{V})$ is a nominal set itself, with action $\pi \cdot [\mathtt{t}]_\alpha \triangleq [\pi \cdot \mathtt{t}]_\alpha$. Equivariant finitely supported relations on $\mathcal{T}^\alpha_\Sigma(\mathcal{V})$, as expected, carry a complete SRA structure.

3.1.3 Further Examples. Due to space constraints, we have to omit further examples. We report some of those in the appendix.

3.2 Expressiveness

What can we do with SRAs? As a warm up, we notice that we can express basic, albeit useful, syntax-based notions.

Definition 3.9. *We say that a relation* a *is: (i)* compatible *if* $\widehat{a} \leq a$*; (ii)* closed under term constructs *if* $\widehat{a} \leq a$*; (iii)* a (pre)congruence *if it is compatible equivalence (preorder); (iv)* closed under (substitution) instances *(CUI) if* $a[\Delta] \leq a$*; (v)* substitutive *if* $a[a] \leq a$*. (vi) We say that* a *satisfies* Leibniz's Substitution Law *[43] if* $\Delta[a] \leq a$*.*

Notice that any compatible relation is reflexive. Moreover, for any a, the relation $a[\Delta]$ (the closure under substitution instances of a) is always CUI. Moreover, compatibility ($\widehat{a} \leq a$) implies Leibniz's law, but the converse is in general false.

Before that, we show how our algebraic account of substitution reveals a rich modal and cohesive structure [68].

3.3 Substitution and Cohesion

Many syntax-based systems build upon the distinction between *complete* and *incomplete* terms [75] or, equivalently, between *closed* and *open* expressions, and operations of restriction to closed terms and extension to open ones. Mathematically, these distinctions and operations are usually left on the background and treated syntactically. The algebraic machinery of SRAs reveals, instead, a rich structure behind these notions.

Let us begin with uncovering the (relational) structure of complete terms. To fix ideas, let us consider first-order terms (the example holds *mutatis mutandis* for second-order syntax) $\mathcal{T}_\Sigma(\mathcal{V})$ ($= \bigcup_\Gamma \mathcal{T}_\Sigma(\Gamma)$). In this case, we say that a term $\mathtt{t} \in \mathcal{T}_\Sigma(\Gamma)$ is *closed* if $\mathtt{t} \in \mathcal{T}_\Sigma(\varnothing)$. Writing $\mathcal{A}$ and $\mathcal{A}_0$ for the quantales of relations over terms and closed terms, respectively, we see that these two algebras are related by the adjoint triple $\iota \dashv \flat \dashv \sharp$, where $\iota, \sharp : \mathcal{A}_0 \to \mathcal{A}$ and $\flat : \mathcal{A} \to \mathcal{A}_0$ are defined by $\flat(\phi) \triangleq \{(\mathtt{t}, \mathtt{s}) \mid \varnothing \vdash \mathtt{t} \mathrel{\phi} \mathtt{s}\}$, $\iota(r)(\Gamma) \triangleq \{(\mathtt{t}, \mathtt{s}) \mid \mathtt{t}\ r\ \mathtt{s}\}$, and $\sharp(r)(\Gamma) \triangleq \{(\mathtt{t}, \mathtt{s}) \mid \mathtt{t}, \mathtt{s} \in \mathcal{T}_\Sigma(\emptyset) \implies \mathtt{t}\ r\ \mathtt{s}\}$. These adjunctions create an (axiomatic) *cohesion* [68], whereby the (co)monadic *modalities* $\Box \triangleq \iota \circ \flat$, $\blacklozenge \triangleq \sharp \circ \flat$ on $\mathcal{A}$ form an adjunction

$\Box \dashv \blacklozenge$ themselves [59]. Making some calculations, we obtain the following explicit definition on relations: $\Gamma \vdash \mathtt{t}\ \Box\phi\ \mathtt{s}$ iff $\varnothing \vdash \mathtt{t}\ \phi\ \mathtt{s}$, and $\Gamma \vdash \mathtt{t}\ \blacklozenge\phi\ \mathtt{s}$ iff $(\varnothing \vdash \mathtt{t}, \mathtt{s} \implies \varnothing \vdash \mathtt{t}\ \phi\ \mathtt{s})$.

Since the modalities $\Box$ and $\blacklozenge$ are themselves obtain through the composition of adjoint maps, they satisfy (co)monadic laws. For instance, $\Box$ being a comonad, it is an *S4-modality* [11], meaning that it is monotone, and satisfies $\Box\phi \subseteq \phi$ and $\Box(\phi\,;\psi) = \Box\phi\,;\Box\psi$. It also distributes over relation converse and composition. Furthermore, we see that whether two terms are related by $\Box\phi$ boils down to the shape of the terms (they must be complete) and the relation ϕ itself (they must be ϕ-related). This is captured by the law $\Box\Delta;\phi;\Box\Delta \subseteq \Box\phi$. Finally, as expected, we have the laws $\Box\Delta_\eta = \bot$ (no variable is a complete term) and $(\Box\phi)[\psi] \subseteq \Box\phi$ (substitution has no effect on complete terms).

3.3.1 Closed Relations. While all the aforementioned laws have been obtained in the context of a specific example, their very structure suggests that the modalities $\Box$ and $\blacklozenge$ should be definable solely in terms of substitution. This is indeed that case, we see that defining $\Box a \triangleq a[\bot]$ and $\blacklozenge a \triangleq a \gg \bot$ we indeed obtain all the aforementioned laws.

Using the modality $\Box$, we obtain an image of the algebra of complete relations within usual SRAs simply by considering *closed relations*, *viz.* relations a such that $a \leq \Box a$ (and thus $\Box a = a$). Closed relations play a crucial role in term evaluation, as the latter acts on complete terms only. Since evaluation and related forms of syntax dynamics are typically defined recursively, as least solutions a^{F} of parametric equations of the form $x = F(x, a)$, we would like to ensure that whenever a is closed, and F is of a suitable form, then a^{F} is closed too.

Lemma 3.10. *Let F be a monotone function on a complete extended with the axioms in Figure 2. We say that F is* closed *if $\Box F(x) \leq F(\Box x)$. If F is closed then $\Box\mu F = \mu x.\Box F(x)$. In particular, $\Box a^{\mathrm{F}} = (\Box a)^{\mathrm{F}}$ and thus $a^{\mathrm{F}} = \Box a^{\mathrm{F}}$ if $a = \Box a$.*

Thinking about F as the extension of a term function to relations, the requirement that F is closed means that if two complete terms are related by $\Box F(a)$, then all the a-related sub-terms involved are complete themselves. Reading all of that as term evaluation, closedness means that evaluating a compound complete term requires to evaluate its complete sub-terms only (cf. with *weak* evaluation, where one does not evaluate under binders precisely because of that).

Finally, we can extend relations on closed terms to open ones through the *open extension* modality [67]. In the case of first-order terms, this is given by the relation $\Diamond\phi$ where $\Gamma \vdash \mathtt{t}\ \Diamond\phi\ \mathtt{s}$ holds iff $\forall\sigma.\ (\forall \mathtt{x} \in \Gamma.\ \sigma(\mathtt{x}) \in \mathcal{T}_\Sigma(\varnothing)) \implies \varnothing \vdash \mathtt{t}[\sigma]\ \phi\ \mathtt{s}[\sigma]$ Abstracting from first-order syntax, we see that the key point of open extension is to behave as the right adjoint $\Box(-[\Delta]) \dashv \Diamond$, which is actually expressible in the language of SRAs thus: $\Diamond a \triangleq \Delta \gg \blacklozenge a$. As we shall see, open extension will play an important role in the definition of bisimilarity.

4 Pointfree Metatheory, I: Reduction Systems

It is finally time to put SRA at work and to develop some metatheory with (and within) them. We first show some examples of metatheory of *reduction systems*. We will not say much, actually, as we shall instead focus on the metatheory of operational semantics and program equivalence.

Definition 4.1. *We say that a relation a is a* reduction *if $\Delta_\eta\,; a = \bot$. This condition [10] ensures nontriviality with respect to (substitution)* instances *of the rule, i.e.term relations obtained from a by instantiating the variables of a-related terms using the same substitution.*

Example 4.2. Given a collection of first-order terms $\mathcal{T}_\Sigma(\mathcal{V})$, a *Term Rewriting System* (TRS) [10, 55] is given by a reduction ϕ in the SRA of relations over $\mathcal{T}_\Sigma(\mathcal{V})$. For example, for the signature of *combinatory logic* [21] $\Sigma \triangleq \{\mathtt{k} : 0, \mathtt{s} : 0, \mathtt{A} : 2\}$, the (closure under extended renaming of the) term relation $\mathtt{x}, \mathtt{y} \vdash \mathtt{A}(\mathtt{A}(\mathtt{k}, \mathtt{x}), \mathtt{y}) \mapsto \mathtt{x}$ is a reduction (and similarly for the corresponding reduction for $\mathtt{s}$). We extend the reduction action of $\mathtt{k}$ from variables to arbitrary terms by considering $\mapsto[\Delta]$, while taking $(\mapsto[\Delta] \vee \Delta)^{\mathbf{H}}$ gives parallel reduction. Sequential reduction can be modelled by considering so-called *derivatives* [86]

Example 4.3. Let us consider the signature of the dependent λ-calculus. We see that β-reduction gives a relation defined by $\Gamma \vdash (\lambda(x : \mathtt{t}).\mathtt{s})\mathtt{u}\ \beta\ \mathtt{s}[\mathtt{u}/\mathtt{x}]$. Notice that $\Delta_\eta\,;\beta = \bot$ and $\beta[\Delta] \leq \beta$. Taking the Howe extension of its reflexive closure $(\beta \cup \Delta)^{\mathbf{H}}$ we obtain *parallel β-reduction* [113] $\Rightarrow_\beta$, which we can inductively characterised thus:

$$\frac{}{\Gamma, \mathtt{x} \vdash \mathtt{x} \Rightarrow_\beta \mathtt{x}} \qquad \frac{\Gamma, \mathtt{x} \vdash \mathtt{t} \Rightarrow_\beta \mathtt{t}' \quad \Gamma \vdash \mathtt{s} \Rightarrow_\beta \mathtt{s}'}{\Gamma \vdash (\lambda \mathtt{x} : \mathtt{u}.\mathtt{t})\mathtt{s} \Rightarrow_\beta \mathtt{t}'[\mathtt{s}'/\mathtt{x}]} \qquad \frac{\Gamma, \mathtt{x} \vdash \mathtt{t}_i \Rightarrow_\beta \mathtt{s}_i}{\Gamma \vdash \lambda \mathtt{x} : \mathtt{t}_1.\mathtt{t}_2 \Rightarrow_\beta \lambda \mathtt{x} : \mathtt{s}_1.\mathtt{s}_2}$$

$$\frac{\Gamma \vdash \mathtt{t} \Rightarrow_\beta \mathtt{t}' \quad \Gamma \vdash \mathtt{s} \Rightarrow_\beta \mathtt{s}'}{\Gamma \vdash \mathtt{t}\mathtt{t}' \Rightarrow_\beta \mathtt{s}\mathtt{s}'} \qquad \frac{\Gamma \vdash \mathtt{t} \Rightarrow_\beta \mathtt{t}' \quad \Gamma, \mathtt{x} \vdash \mathtt{s} \Rightarrow_\beta \mathtt{s}'}{\Gamma \vdash \Pi(\mathtt{x} : \mathtt{t}).\mathtt{s} \Rightarrow_\beta \Pi(\mathtt{x} : \mathtt{t}').\mathtt{s}'}$$

Finally, taking $(\beta \vee \beta^\circ)^{\mathbf{H}*}$ gives *β-convertibility* $=_\beta$.

Abstracting from the previous example, we define the *parallel reduction* over a as $a^{\Rightarrow} \triangleq (\Delta \vee a[\Delta])^{\mathbf{H}}$ and convertibility as $a^{\simeq} \triangleq (a \vee a^\circ)^{\mathbf{H}*}$. Standard algebraic SRA calculations show that $a^{\simeq} \triangleq (a^{\Rightarrow} \vee a^{\Rightarrow\circ})^*$.

A central problem in rewriting is the one of determining consistency of convertibility, which often reduces to prove the so-called *Church-Rosser property* [10].

Definition 4.4. *We say that a relation a has the* Church-Rosser property *if $(a \vee a^\circ)^* = a^*\,; a^{*\circ}$, and that it has the* diamond property *if $a^\circ\,; a \leq a\,; a^\circ$. We say that a is* confluent *if a^* has the diamond property.*[8]

Easy algebraic calculations show that if a has the diamond property, then it is confluent; and that a relation has the Church-Rosser property iff it is confluent [110, 111]. In light of that, to prove that $a^{\Rightarrow}$ has the Church-Rosser it is sufficient to prove that $a^{\Rightarrow}$ has the diamond property. Using the language of SRAs, we can algebraically express an easy-to-check 'semantic' notion of *orthogonality* (cf. [**?**]) and calculationally infer confluence from it.

Theorem 4.5 ([35])**.** *A reduction a is* orthogonal *if $a[\Delta]^\circ\,; a[\Delta] \leq \Delta$ and $a[\Delta]^\circ\,; \widehat{a[\Delta]^{\Rightarrow}} \leq a^\circ[a^{\Rightarrow}]$. If a is orthogonal, then $a^{\Rightarrow}$ is diamond, and thus confluent.*

[8] Even if we will not deal with termination, we mention, for the pure sake of beauty, the particularly pleasant laws [26] expressing that a relation a is *inductive* — $a \to x \leq x \implies \top \leq x$ — and *well-founded* — $x \leq x\,; a \implies x \leq \bot$.

The first condition in orthogonality states that substitution instance of (ground) reduction is deterministic. The second condition asserts that even reducing substerms of a redex does not destroy the redex structure and, the new nested redexes it creates can be uniformly reduced.

Example 4.6. The relation β is orthogonal: $\Gamma \vdash \mathtt{t}[\mathtt{s}/\mathtt{x}]\ \beta^{\circ}\ (\lambda\mathtt{x}.\mathtt{t})\mathtt{s}$ and $\Gamma \vdash (\lambda\mathtt{x}.\mathtt{t})\mathtt{s} \Rrightarrow_{\beta} (\lambda\mathtt{x}.\mathtt{t}')\mathtt{s}'$ entail $\Gamma \vdash \mathtt{t}[\mathtt{s}/\mathtt{x}] \Rrightarrow_{\beta} \mathtt{t}'[\mathtt{s}'/\mathtt{x}]$ and $\Gamma \vdash \mathtt{t}'[\mathtt{s}'/\mathtt{x}]\ \beta^{\circ}\ (\lambda\mathtt{x}.\mathtt{t}')\mathtt{s}'$, since $\Rrightarrow_{\beta}$. This follows since, in general, $a^{\Rrightarrow}$ is substitutive ($a^{\Rrightarrow}[a^{\Rrightarrow}] \leq a^{\Rrightarrow}$). By Theorem 4.5, we conclude confluence.

While our notion of orthogonality is typically easy to check, it is not a genuine local conditions, as it speaks of $a^{\Rrightarrow}$. In the next section, we will give a finer, completely local conditions and show that such condition implies Theorem 4.5.

5 Pointfree Metatheory: Operational Semantics

Operational semantics involves diverse 'syntactic' notions: program, values, context, lazy and eager evaluation, reduction strategies, and many others, that are typically left, implicitly or explicitly, to term representations. Much of these notions can be uniformly understood following (the informal account by) Martin-Löf [75] (see also the nice explanation given in [**?**]). Accordingly, term constructs are divided into *constructors* and *destructors* [**?**]: the former construct pieces of structured data; the latter consume them. Arguments of a destructor are furthermore divided into *major* — or *eager* — *minor* — or *lazy*. Terms whose outermost operator is a constructor (resp. destructor) are *introduction forms* (resp. *elimination forms*). Computation is triggered when there is an elimination form with introduction forms as major arguments: this is the so-called *Gentzen's Inversion Principle* [36].[9]

Closed terms — *programs* — are evaluated until a *canonical form* — *value* — is reached. In a lazy discipline, canonical forms are identified with introduction forms (for example, we do not reduce under λs); in eager disciplines, other notions of canonical form may be considered (*e.g.* introduction forms whose arguments are canonical themselves). Languages with 'eager canonical forms' can be turned into lazy ones by adopting a *reduced syntax* [67], or *fine-grain* syntax [70]. For simplicity, we will also identify canonical forms with introduction forms.

Finally, evaluation proceeds thus: if a closed term is an introduction form (a value), then just return it. Otherwise the term is an elimination form; evaluate its major sub-terms — which has to be closed, otherwise the evaluation fails — until an introduction form is reached. At this point, by Inversion Principle, if the introduction forms correspond to the elimination form to be evaluated, a true computation step is performed and then evaluation is resumed.

5.1 Algebraization of Operational Semantics

All these notions can be easily expressed, algebraically, in the language of SRA. The crucial gadget to that is the one of an *operational decomposition*.

[9] Notice that following this view, the purpose of a type discipline is precisely to ensure that elimination forms interact only with their corresponding introduction forms.

Definition 5.1. *Given a SRA, an* operational decomposition *on it consists of weakly unital orthogonal morphisms* $\overline{\cdot}$ *and* $\langle\, \cdot\,_1\rangle \cdot_2$ *such that:*

1. $\widetilde{a} = \overline{a} \vee \langle a, a\rangle$.
2. $\cdot\,[b]$ *distributes over* $\overline{\cdot}$ *and* $\langle\, \cdot\,_1,\ \cdot\,_2\rangle$ *(cf. right tensorial strength).*
3. $\Box$ *distributes over* $\langle\, \cdot\,, b\rangle$ *(i.e.* $\Box\langle a, b\rangle \leq \langle \Box a, b\rangle$*).*

The intuitive meaning of an operational decomposition should be clear. Both $\overline{a}$ and $\langle a, b\rangle$ refine strict compatible refinement by restricting outermost operators to constructors, in the case of $\overline{a}$, and destructors, in the case of $\langle a, b\rangle$. Furthermore, $\langle a, b\rangle$ requires *major* subterms to be pairwise a-related and all the other (*minors*) to be pairwise b-related. Using these operations, we define the co-equivalences of introduction and elimination forms as $\overline{\Delta}$ and $\langle \Delta, \Delta\rangle$, and the co-equivalence of *values* as complete introduction forms: $\Delta_{\kappa} \triangleq \Box\overline{\Delta}$.

Notice that orthogonality gives $\overline{a}\ ;\ \langle b, c\rangle \leq \bot$, meaning that constructors and destructors are disjoint. If we identify canonical forms with introduction forms, as we do, then orthogonality implies that terms with an outermost constructors are always values (*i.e.* evaluation must be lazy, or weak). While this is always the case in call-by-name semantics, it is not so in call-by-value one: for example, a pair $\mathtt{pair}(\mathtt{t}, \mathtt{s})$ is a value if both $\mathtt{t}$ and $\mathtt{s}$ are. We can easily overcome this issue by presenting syntax in *reduced* or *fine-grain* form [67, 70]. Finally, distributivity of $\Box$ over $\langle\, \cdot\,, b\rangle$ ensures that major arguments of a closed term are closed themselves.

5.1.1 Inversion Principle. Let us now algebrize Gentzen's Inversion Principle. Since we will extensively relate elimination forms through their major arguments, we introduce the abbreviating $\langle a\rangle \triangleq \langle a, \Delta\rangle$. Intuitively, two elimination forms are related by $\langle a\rangle$ if they same outermost destructor and minor arguments, and pairwise a-related major arguments.

Definition 5.2. *We say that a relation a satisfies* Gentzen's Inversion Principle *(GIP) if $a \leq \langle\overline{\Delta}\rangle\ ;\ a$ (and thus $a = \langle\overline{\Delta}\rangle\ ;\ a$).*

Notice that all reductions met so far, as well as many others, satisfy (GIP). A stronger version of this principle — dubbed *Strong Gentzen's Inversion Principle* (SGIP) — can obtained by introducing a *domain* operation dom giving, for each relation a, the co-equivalence representing the domain of a [26, 31, 105]. Assuming dom, we can express (SGIP) as $\mathrm{dom}(a) = \langle\overline{\Delta}\rangle$. This equation, which states that any elimination form whose major arguments are introduction forms is reducible, holds on, *e.g.*, β, although it already fails when moving to simple extensions, such as PCF, due to stuck terms (*e.g.* $\mathtt{app}(\mathtt{z}, \mathtt{z})$). Typing disciplines, however, typically ensure the validity of SGIP.

5.1.2 Examples. A large family of examples can be obtained by splitting (first-order, binding, nominal, *etc.*) signatures Σ into two (first-order, binding, nominal, *etc.*) *disjoint* signatures Σ_{intro} and Σ_{elim} containing constructors and destructors, respectively. Notice that, as just discussed, we require that constructors and destructors are disjoint.

Next, we refine the notion of arity of a destructor so as to specify which operands are *major* and *minor* arguments. This can be done by labelling specific notions of arity. For example, in a binding

signature with $|\mathbf{op}| = (k_1, \ldots, k_n)$, we label those k_js corresponding to major positions, provided that these do *not* bind variables. Finally, we use a 'corresponding relation' $\sim \subseteq \Sigma_{\mathsf{elim}} \times \Sigma_{\mathsf{intro}}$ relating destructors with their corresponding constructors. As already mentioned, type systems can be understood as way to constraining term formation according to $\sim$.

Example 5.3. We refine the signature of the λ-calculus with dependent types and natural numbers thus:
$\Sigma_{\mathsf{intro}} \triangleq \{\mathtt{lam} : (1), \mathtt{Pi} : (0,1), \mathtt{refl} : (0), \mathtt{nat} : (), \mathtt{z} :: (), \mathtt{succ} : (0)\}$, $\Sigma_{\mathsf{elim}} \triangleq \mathtt{app} : (\underline{0}, 0), \mathtt{eqrec} : (\underline{0}, 0), \mathtt{natrec} : (\underline{0}, 0, 2)\}$, where we underline major positions. The relation $\sim$ is defined by: $\mathtt{app} \sim \mathtt{lam}$, $\mathtt{eqrec} \sim \mathtt{refl}$, $\mathtt{natrec} \sim \mathtt{z}$, and $\mathtt{natrec} \sim \mathtt{succ}$. Reduction is then determined by the scheme: $(\mathtt{app}(\mathtt{lam}(-), -), \mathtt{eqrec}(\mathtt{refl}(-), -), \mathtt{natrec}(\mathtt{z}, -, -)$, and $\mathtt{natrec}(\mathtt{succ}(-), -, -)$. Other interactions between destructors and constructors following this pattern give so-called stuck terms.

Notice that the choice of major premises in a destructors determines the evaluation order of a term. In the example above, for instance, the destructor $\mathtt{app} : (\underline{0}, 0)$ stipulates that the evaluation of a term $\mathtt{app}(\mathtt{t}, \mathtt{s})$ proceeds by evaluating $\mathtt{t}$ until a corresponding introduction form is reached, *viz.* $\mathtt{lam}(\mathtt{x}.\mathtt{t}')$, and then performing β-reduction according Gentzen's principle — hence reducing $\mathtt{app}(\mathtt{lam}(\mathtt{x}.\mathtt{t}'), \mathtt{s})$ to $\mathtt{t}'[\mathtt{s}/\mathtt{x}]$ — *without* evaluating $\mathtt{s}$. In this sense, our syntax underpins a *call-by-name* evaluation strategy.

Example 5.4. The *call-by-value* λ-calculus is given by the signatures $\Sigma_{\mathsf{intro}} \triangleq \{\mathtt{lam} : (1)\}$ and $\Sigma_{\mathsf{elim}} \triangleq \mathtt{app} : (\underline{0}, \underline{0})\}$. Accordingly, the evaluation of $\mathtt{app}(\mathtt{t}, \mathtt{s})$ requires to evaluate *both* $\mathtt{t}$ and $\mathtt{s}$.

Constructing examples of operational decomposition is now straightforward. On binding signature, for instance, we define $\overline{\phi}$ and $\langle \phi, \psi \rangle$ thus, where, for a destructor $\mathbf{op} : (k_1, \ldots, k_n)$, we have $m_{\mathbf{op}}(i, \phi, \psi) \triangleq \phi$ if k_i major, and $m_{\mathbf{op}}(i, \phi, \psi) \triangleq \psi$, otherwise.

$$\frac{\Gamma, \vec{\mathtt{x}}_1 \vdash \mathtt{t}_1 \ \phi \ \mathtt{s}_1 \quad \cdots \quad \Gamma, \vec{\mathtt{x}}_n \vdash \mathtt{t}_n \ \phi \ \mathtt{s}_n \quad \mathbf{op} : (k_1, \ldots, k_n) \in \Sigma_{\mathsf{intro}}}{\Gamma \vdash \mathbf{op}(\vec{\mathtt{x}}_1.\mathtt{t}_1, \ldots, \vec{\mathtt{x}}_n.\mathtt{t}_n) \ \overline{\phi} \ \mathbf{op}(\vec{\mathtt{x}}_1.\mathtt{s}_1, \ldots, \vec{\mathtt{x}}_n.\mathtt{s}_n)}$$

$$\frac{(\forall 1 \leq i \leq n) \ \Gamma, \vec{\mathtt{x}}_i \vdash \mathtt{t}_i \ m_{\mathbf{op}}(i, \phi, \psi) \ \mathtt{s}_i \quad \mathbf{op} : (k_1, \ldots, k_n) \in \Sigma_{\mathsf{elim}}}{\Gamma \vdash \mathbf{op}(\vec{\mathtt{x}}_1.\mathtt{t}_1, \ldots, \vec{\mathtt{x}}_n.\mathtt{t}_n) \ \langle \phi, \psi \rangle \ \mathbf{op}(\vec{\mathtt{x}}_1.\mathtt{s}_1, \ldots, \vec{\mathtt{x}}_n.\mathtt{s}_n)}$$

Structural Recursion, Revisited. The refinement of $\widetilde{\cdot}$ (and, consequentely, of $\widehat{\cdot}$) into the operations $\overline{\cdot}$ and $\langle \cdot, \cdot \rangle$ opens the door to a number of refinements of the operation $\cdot^{\mathbf{H}}$, each corresponding to various forms of structural recursion on, *e.g.*, elimination forms, evaluation context, *etc.* In light of applications (and due to space constraint), here we consider only the operator $\cdot^{\mathbf{F}}$, which captures the recursive pattern needed to define term evaluation: where $a^{\mathbf{H}}$ relates variables, $a^{\mathbf{F}}$ relates introduction forms; and where $a^{\mathbf{H}}$ recursively applies on the operands of a compound term, $a^{\mathbf{H}}$ recursively applies on the *major* operands of an elimination form. Consequently, we see that $a^{\mathbf{F}}$ arises as the least solution (which indeed exists) to the equation $x = (\overline{\Delta} \vee \langle x \rangle) \ ; a$, (cf. with $x = \widehat{x} \ ; a$). It is easy to recast $\cdot^{\mathbf{F}}$ via initiality on $\mathcal{A}^{\dagger}$.

Big-Step Semantics. Let us now algebrize the informal notion of evaluation *á la* Martin-Löf. Let us fix a SRA $\mathcal{A}$ with an operational decomposition, and agree that, from now on, any reduction tacitly satisfies (GIP). Notice, in particular, that this implies $\overline{x} \ ; a \leq \bot$, for any reduction a.

Definition 5.5. *Given a reduction a, we define the* one-step evaluation *induced by a as* $a^{E} \triangleq (a \vee \overline{\Delta})^{\mathbf{F}}$.

Intuitively, $a^{\mathbf{E}}$ either relates introduction forms with themselves or it recursively evaluates major arguments of an elimination form, then a-reducing the result obtained. In this sense, we see that it refines parallel reduction following the reduction strategy imposed by the operational decomposition.

We now define evaluation by iterating $a^{\mathbf{E}}$ until a canonical form is reached.

Definition 5.6. *Given a reduction a, we define the* evaluation — *or* big-step semantics — *induced by a as* $a^{\Downarrow} \triangleq a^{E*} \ ; \overline{\Delta}$.

As expected, we can characterise $a^{\Downarrow}$ inductively.

Proposition 5.7. *Let a be a reduction. Then $a^{\Downarrow}$ is the least solution to the equation $x = \overline{\Delta} \vee \langle x \rangle \ ; a \ ; x$. Furthermore, $\Box a^{\Downarrow} = (\Box a)^{\Downarrow}$. In particular, if a is closed, then $a^{\Downarrow}$ is closed too, and it is the least solution of the equation $x = \Delta_{\kappa} \vee \langle x \rangle \ ; a \ ; x$.*

Proof Sketch. Observe that $a^{\mathbf{E}}$ is the least solution to the equation $x = \overline{\Delta} \vee \langle x \rangle ; a$, since $(\overline{\Delta} \vee \langle x \rangle) ; (a \vee \overline{\Delta}) = (\overline{\Delta} ; a) \vee \overline{\Delta} \vee (\langle x \rangle ; a) \vee (\langle x \rangle ; \overline{\Delta})$ $= \overline{\Delta} \vee \langle x \rangle \ ; a$. As a consequence, $\Box a^{\mathbf{E}} = (\Box a)^{\mathbf{E}}$. In particular, if a is closed, then $a^{\mathbf{E}}$ is closed too, and it is the least solution of the equation $x = \Delta_{\kappa} \vee \langle x \rangle \ ; a$. Then exploit the least fixed point characterisations of a^{*}. □

Example 5.8. Let us consider the call-by-value λ-calculus (Example 5.4), and let β_v be call-by-value β-reduction $(\Gamma \vdash (\lambda \mathtt{x}.\mathtt{t}) \mathtt{v} \ \beta \ \mathtt{t}[\mathtt{v}/\mathtt{x}])$. Then, $\Box \beta^{\Downarrow}$ coincides with the usual big-step semantics $\Downarrow_{\beta_v}$.

Now the main metatheorem of this section: evaluation is deterministic whenever its underlying reduction is.

Theorem 5.9 (Determinism). *Let a be a closed reduction. Then: $a \ ; a^{\circ} \leq \Delta \implies a^{\Downarrow \circ} \ ; a^{\Downarrow} \leq \Delta_{\kappa}$*

Proof Sketch. The proof is composed of different parts. First, we show that GIP implies the well-knonw *inversion lemma* [88], which is relationally given by $\langle b \rangle ; a^{\Downarrow} \leq \langle b ; a^{\Downarrow} \rangle ; a^{\Downarrow}$. We use the latter to show that, under the assumption $a \ ; a^{\circ} \leq \Delta$, we have $a^{\circ} \ ; a^{\Downarrow} \leq a^{\Downarrow}$. We then prove that $a^{\circ} \ ; a^{\Downarrow} \leq a^{\Downarrow}$ implies $a^{\Downarrow \circ} \ ; a^{\Downarrow} \leq \Delta_{\kappa}$. □

6 Pointfree Metatheory: Program Equivalence

In this last section, we outline a further application of SRA to the metatheory of *program equivalence*.

The literature offers many notions of equivalence and approximation, notable ones include *contextual equivalence* and *approximation* [84], *Kleene equivalence* and *approximation* [98], and *(bi)simulation*-based equivalence [80, 87], in particular in the form of *applicative (bi)similarity* [2]. While all of these can be studied through SRAs, due to space constraints, we focus on applicative (bi)similarity being one of the most meta-theoretically challenging.

6.1 Applicative (Bi)Similarity

(Applicative) similarity is a *coinductively* defined preorder on complete terms. Intuitively, we say that a complete $\mathtt{t}$ is similar to $\mathtt{t}'$ if whenever $\mathtt{t}'$ ($a^{\Downarrow}$)-evaluates to a canonical term $\mathtt{v}'$, then also $\mathtt{t}$ evaluates to a canonical term $\mathtt{v}$, and $\mathtt{v}$ and $\mathtt{v}'$ have the same outermost constructor (*e.g.*they are both lambdas, pairs, *etc.*) with pairwise

similar operands. Similarity is defined coinductively as the greatest simulation.

We can then 'semi-formally' say that similarity is the largest complete relation sim such that, whenever $\mathtt{t}$ sim $\mathtt{t}'$ (with $\mathtt{t}, \mathtt{t}'$ complete) and $\mathtt{t}'\ a^{\Downarrow}\ \mathtt{v}'$, then $\mathtt{t}\ a^{\Downarrow}\ \mathtt{v}$ and $\mathtt{v}\ \overline{\Diamond\mathsf{sim}}\ \mathtt{v}'$. Notice that even if similarity is defined on complete terms, we need to take its open extension when it comes to relate sub-terms of introduction forms, as such sub-terms may not be complete themselves. It is an easy exercise to check that, on concrete examples, we obtain Abramsky's applicative similarity [2] and its generalisation by Howe [54].

Definition 6.1. *Let b be a closed reduction. We define $B_b(a)$ as $(b^{\gtrsim}\,;\overline{a}) \leftarrow b^{\gtrsim}$, and say that a is a b-*simulation *if $a \le B_b(\Diamond a)$, i.e. $a\,; b^{\gtrsim} \le b^{\gtrsim}\,; \overline{\Diamond a}$.*

Let b be fixed from now on, so that we can omit subscripts. It is easy to see that B is monotone, and that $B(a)$ is always complete, *i.e.* $\Box B(a) = B(a)$. This means that the equation $x = B(\Diamond x)$ has a greatest solution, which we denote by $b^{\gtrsim}$ and call *(b-)similarity*. Notice that $b^{\gtrsim}$ is complete (*i.e.* $\Box b^{\gtrsim} = b^{\gtrsim}$), and comes with a corresponding *coinduction proof principle*: $a \le B(a) \implies a \le b^{\gtrsim}$. Using the latter, we see that $b^{\gtrsim}$ is a preorder on closed terms: $\Box\Delta \le b^{\gtrsim}$ and $b^{\gtrsim}\,; b^{\gtrsim} \le b^{\gtrsim}$. Furthermore, its (open) extension $\Diamond b^{\gtrsim}$ to incomplete terms — known as *open similarity* — still admits a coinductive characterisation as the greatest solution of the equation $x = \Diamond B(x)$. From that follows that $\Diamond b^{\gtrsim}$ is a preorder.

Howe's Method. The main meta-theoretical result on $\Diamond b^{\gtrsim}$ is that it is *compatible* and *substitutive*, a result typically proved using *Howe's method* [54, 90]. We now outline an algebraic and language-independent proof of compatibility and substitutivity.

Given our 'dualised' definition of simulation it is more convenient to use a dual version of the Howe extension operator — *op-Howe extension* — to be applied to closed relations: $a^{\S} \triangleq (\Diamond a)^{\mathbf{H}\circ}$. By the general theory of SRAs, we see that, for any closed preorder a, $a^{\S}$ is *reflexive*, *substitutive*, *compatible* ($\widehat{a^{\S}} \le a^{\S}$), and satisfies the so-called *quasi-transitivity* law: $\Diamond a\,; a^{\S} \le a^{\S}$.

To prove compatibility, we need to spell out an important hypothesis, one that is hardly detectable without the algebraic vocabulary of SRAs and that express a fundamental principle of language design [48]: *Gentzen Conservation Principle*.

Definition 6.2 (Gentzen Conservation Principle)**.** *We say that a closed reduction a is satisfied* Gentzen Conservation Principle *(GCP) if for any compatible relation x, we have $\langle\widehat{x}, x\rangle\,; a \le a\,; x[x]$.*

Albeit cryptic at first, Definition 6.2 is deeply connected with the proof-theoretic principle of *harmony* [27, 103], as well as with Plotkin's account of a *structural rule* [99]. The equation $\langle\widehat{x}, x\rangle; a \le a; x[x]$, an embryonic version of which is due to Lassen [67], states two facts: *(i)* that the behaviour of a reduction a is determined by the outermost operator of the term to which it is applied, along with the outermost shape of its operands, being parametric in all the rest; *(ii)* that computation proceeds by substitution.[10] To see that, consider the example of the call-by-value λ-calculus: if two terms are related by $\langle\widehat{\phi}, \phi\rangle\,;\beta_v$, then we must have $\varnothing \vdash (\lambda\mathtt{x.p})\mathtt{w}\ \langle\widehat{\phi}, \phi\rangle\ (\lambda\mathtt{x.u})\mathtt{v}$, with $\varnothing \vdash \mathtt{w}\ \phi\ \mathtt{v}$ and $\mathtt{x} \vdash \mathtt{p}\ \phi\ \mathtt{u}$. Those give $\mathtt{p}[\mathtt{w}/\mathtt{x}]\ \phi[\phi]\ \mathtt{u}[\mathtt{v}/\mathtt{x}]$ which, together with $(\lambda\mathtt{x.p})\mathtt{w}\ \beta_v\ \mathtt{p}[\mathtt{w}/\mathtt{x}]$, precisely gives $\langle\widehat{\phi}, \phi\rangle;\beta_v \le (\beta_v;\phi[\phi])$.

Lemma 6.3 (Key Lemma)**.** *Let a be a closed reflexive and transitive simulation. If b satisfies (GCP), then $a^{\S} \le \Diamond B(a^{\S})$.*

Sketch. First, an easy calculation shows that $a^{\S} \le \Diamond B(a^{\S})$ is equivalent to $b^{\gtrsim} \le \Box a^{\S} \to (b^{\gtrsim}\,; \widehat{a^{\S}})$. The latter is proved by fixed point induction on $b^{\gtrsim}$. The proof is then given by a (long) equational calculation. □

Theorem 6.4. *$\Diamond b^{\gtrsim}$ is substitutive and compatible.*

Proof Sketch. By Lemma 6.3, $b^{\gtrsim\S}$ is an open simulation, hence $b^{\gtrsim\S} \le \Diamond b^{\gtrsim}$, by coinduction. Since $\Diamond b^{\gtrsim} \le b^{\gtrsim\S}$, we conclude $b^{\gtrsim\S} = \Diamond b^{\gtrsim}$ and thus compatibility of substitutivity of $\Diamond b^{\gtrsim}$. □

Finally, we can easily rephrase the so-called "transitive-closure trick" [90] to obtain congruence of *(applicative) bisimilarity*, which is defined as $\nu x.\Diamond x \wedge B(\Diamond x^{\circ})^{\circ}$.

6.1.1 Confluence, Revisited. A remarkable property of having both (GCP) and (GIP) is that they not only ensure congruence of (bi)similarity, but also confluence of parallel reduction, thereby unifying different metatheoretical results via a natural local conditions.

Lemma 6.5. *Let a be a reduction. If a satisfies (GCP) and (GIP), then $a[\Delta]^{\circ}\,; \widehat{a[\Delta]^{\Rightarrow}} \le a^{\circ}[a^{\Rightarrow}]$.*

Proof Sketch. Observe that $\overline{x}\,; a^{\Rightarrow} \le \overline{x\,; a^{\Rightarrow}}$. □

Theorem 6.6. *Let a be a substitutive and deterministic reduction. If a satisfies (GCP) and (GIP), then: (i) $a^{\Rightarrow}$ is confluent; (ii) $a^{\Downarrow}$ is deterministic; (iii) $a^{\gtrsim}$ is compatible and substitutive.*

7 Related Work and Conclusion

Related Work. The idea of employing a fully fledged algebra of relations to abstract syntactic arguments is due, to the best of the author's knowledge, to Lassen [67]. Following a previous work by Gordon [40, 41], Lassen introduced various syntax-based operations on specific λ-calculi. Many crucial operations, however, are not present in Lassen's system, and that prevented the development of algebraic accounts of reduction and evaluation, thereby leaving reasoning markedly syntactic. Even when relational operations are available, the most significant part of proofs has to be carried out syntactically. Gordon [42] defined operations similar to Lassen's ones in the context of the ς-calculus [1]. That work can be seen as a specific example of our framework.

While other operations on syntax relations have been introduced — on specific calculi and in rather ad hoc ways — along the years [22, 23, 34, 69], the first systematic account of syntax relations, independently of syntax, has been given in [35].

Conclusion. This work constitute a first, foundational step in the research programme outlined in [35]. Other results in that direction have already been proven, but omitted due to space constraints. Much work is currently in progress, particularly concerning algebraic analysis of termination and typing.

[10] This is the case for many, but not all languages. One can consider alternatives to the substitution model of computation, and formulate GCP-like laws of the form $\langle\widehat{x}, x\rangle\,; a \le a\,; F(x)$, with F describing how computation is performed.

References

[1] Martín Abadi and Luca Cardelli. 1996. *A Theory of Objects*. Springer. doi:10.1007/978-1-4419-8598-9

[2] S. Abramsky. 1990. The Lazy Lambda Calculus. In *Research Topics in Functional Programming*, D. Turner (Ed.). Addison Wesley, 65–117.

[3] Luca Aceto, Wan Fokkink, and Frits Vaandrager. 2001. Structural operational semantics. In *Handbook of Process Algebra*, J. A. Bergstra, A. Ponse, and S. A. Smolka (Eds.). Elsevier, 197–292.

[4] Peter Aczel. 1977. An Introduction to Inductive Definitions. In *HANDBOOK OF MATHEMATICAL LOGIC*, Jon Barwise (Ed.). Studies in Logic and the Foundations of Mathematics, Vol. 90. Elsevier, 739–782. doi:10.1016/S0049-237X(08)71120-0

[5] Peter Aczel. 1978. A general Church-Rosser theorem. *Draft, Manchester* (1978).

[6] Hajnal Andréka, Judit X. Madarász, and István Németi. 2004. Algebras of relations of various ranks, some current trends and applications. *Journal on Relational Methods in Computer Science* 1 (2004), 27–49. Special volume (JoRMiCS).

[7] Franz Baader and Tobias Nipkow. 1998. *Term rewriting and all that*. Cambridge University Press.

[8] H.P. Barendregt. 1984. *The lambda calculus: its syntax and semantics*. North-Holland.

[9] Henk Barendregt, Wil Dekkers, and Richard Statman. 2013. *Lambda Calculus with Types*. Cambridge University Press. doi:10.1017/CBO9781139032636

[10] M. Bezem, J.W. Klop, E. Barendsen, R. de Vrijer, and Terese. 2003. *Term Rewriting Systems*. Cambridge University Press.

[11] Patrick Blackburn, Maarten de Rijke, and Yde Venema. 2001. *Modal Logic*. Cambridge Tracts in Theoretical Computer Science, Vol. 53. Cambridge University Press.

[12] Filippo Bonchi, Fabio Gadducci, Aleks Kissinger, Pawel Sobocinski, and Fabio Zanasi. 2022. String Diagram Rewrite Theory I: Rewriting with Frobenius Structure. *J. ACM* (2022).

[13] Filippo Bonchi, Fabio Gadducci, Aleks Kissinger, Pawel Sobocinski, and Fabio Zanasi. 2022. String Diagram Rewrite Theory II: Rewriting with Symmetric Monoidal Structure. *Mathematical Structures in Computer Science* (2022).

[14] Filippo Bonchi, Fabio Gadducci, Aleks Kissinger, Pawel Sobocinski, and Fabio Zanasi. 2022. String Diagram Rewrite Theory III: Confluence with and without Frobenius. *Mathematical Structures in Computer Science* (2022).

[15] Peio Borthelle, Tom Hirschowitz, and Ambroise Lafont. 2020. A Cellular Howe Theorem. In *Proc. of LICS 2020*, Holger Hermanns, Lijun Zhang, Naoki Kobayashi, and Dale Miller (Eds.). ACM, 273–286. doi:10.1145/3373718.3394738

[16] Stanley Burris and H. P. Sankappanavar. 1981. *A Course in Universal Algebra*. Springer, New York.

[17] A. Burroni. 1993. Higher-dimensional word problems with applications to equational logic. *Theoretical Computer Science* 115, 1 (1993), 43–62.

[18] Rod M. Burstall. 1969. Proving Properties of Programs by Structural Induction. *Comput. J.* 12, 1 (1969), 41–48. doi:10.1093/COMJNL/12.1.41

[19] J.H. Conway. 2012. *Regular Algebra and Finite Machines*. Dover Publications, Incorporated.

[20] Andrea Corradini, Reiko Heckel, and Ugo Montanari. 2002. Compositional SOS and beyond: A coalgebraic view of open systems. *Theoretical Computer Science* 280, 1–2 (2002), 163–192.

[21] H.B. Curry and R. Feys. 1958. *Combinatory Logic*. Number v. 1 in Combinatory Logic. North-Holland Publishing Company.

[22] Ugo Dal Lago and Francesco Gavazzo. 2022. A relational theory of effects and coeffects. *Proc. ACM Program. Lang.* 6, POPL (2022), 1–28. doi:10.1145/3498692

[23] U. Dal Lago, F. Gavazzo, and P.B. Levy. 2017. Effectful applicative bisimilarity: Monads, relators, and Howe's method. In *Proc. of LICS 2017*. 1–12.

[24] B.A. Davey and H.A. Priestley. 1990. *Introduction to lattices and order*. Cambridge University Press.

[25] Jr. David A. Vogan. 2005. Review of "Lectures on the Orbit Method," by A. A. Kirillov. *Bull. Amer. Math. Soc.* 42, 4 (2005). doi:10.1090/S0273-0979-05-01065-7

[26] Henk Doornbos, Roland Carl Backhouse, and Jaap van der Woude. 1997. A Calculational Approach to Mathematical Induction. *Theor. Comput. Sci.* 179, 1-2 (1997), 103–135. doi:10.1016/S0304-3975(96)00154-5

[27] Michael Dummett. 1991. *The Logical Basis of Metaphysics*. Harvard University Press, Cambridge.

[28] Marcelo P. Fiore. 2008. Second-Order and Dependently-Sorted Abstract Syntax. In *Proc. of LICS 2008*. IEEE Computer Society, 57–68. doi:10.1109/LICS.2008.38

[29] Marcelo P. Fiore, Gordon D. Plotkin, and Daniele Turi. 1999. Abstract Syntax and Variable Binding. In *14th Annual IEEE Symposium on Logic in Computer Science, Trento, Italy, July 2-5, 1999*. IEEE Computer Society, 193–202. doi:10.1109/LICS.1999.782615

[30] Marcelo P. Fiore and Sam Staton. 2006. A congruence rule format for name-passing process calculi from mathematical structural operational semantics. In *Proceedings of the 21st Annual IEEE Symposium on Logic in Computer Science (LICS 2006)*. IEEE Computer Society, 49–58.

[31] Peter J. Freyd and Andre Scedrov. 1990. *Categories, allegories*. North-Holland mathematical library, Vol. 39. North-Holland.

[32] Marcelo F. Frias. 2002. *Fork Algebras in Algebra, Logic and Computer Science*. Advances in Logic, Vol. 2. World Scientific. doi:10.1142/4899

[33] Murdoch Gabbay and Andrew M. Pitts. 1999. A New Approach to Abstract Syntax Involving Binders. In *14th Annual IEEE Symposium on Logic in Computer Science, Trento, Italy, July 2-5, 1999*. IEEE Computer Society, 214–224. doi:10.1109/LICS.1999.782617

[34] Francesco Gavazzo. 2018. Quantitative Behavioural Reasoning for Higher-order Effectful Programs: Applicative Distances. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2018, Oxford, UK, July 09-12, 2018*. 452–461.

[35] Francesco Gavazzo. 2023. Allegories of Symbolic Manipulations. In *Proc. of LICS*. 1–15. doi:10.1109/LICS56636.2023.10175807

[36] Gerhard Gentzen. 1935. Untersuchungen Über Das Logische Schliessen. I. *Mathematische Zeitschrift* 35 (1935), 176–210.

[37] Jean-Yves Girard, Paul Taylor, and Yves Lafont. 1989. *Proofs and types*. Cambridge University Press, USA.

[38] Joseph A. Goguen, James W. Thatcher, Eric G. Wagner, and Jesse B. Wright. 1977. Initial Algebra Semantics and Continuous Algebras. *J. ACM* 24, 1 (1977), 68–95.

[39] Sergey Goncharov, Stefan Milius, Lutz Schröder, Stelios Tsampas, and Henning Urbat. 2023. Towards a Higher-Order Mathematical Operational Semantics. *Proc. ACM Program. Lang.* 7, POPL (2023), 632–658. doi:10.1145/3571215

[40] A.D. Gordon. 1994. A Tutorial on Co-induction and Functional Programming. In *Workshops in Computing*. Springer London, 78–95.

[41] A.D. Gordon. 1995. Bisimilarity as a theory of functional programming. In *Proc. of MFPS 1995 (Electronic Notes in Theoretical Computer Science, Vol. 1)*, Stephen D. Brookes, Michael G. Main, Austin Melton, and Michael W. Mislove (Eds.). Elsevier, 232–252. doi:10.1016/S1571-0661(04)80013-6

[42] Andrew D. Gordon. 1998. Operational Equivalences for Untyped and Polymorphic Object Calculi. In *Higher Order Operational Techniques in Semantics*, Andrew D. Gordon and Andrew M. Pitts (Eds.). Cambridge University Press, Cambridge, UK.

[43] David Gries and Fred B. Schneider. 1993. *A Logical Approach to Discrete Math*. Springer. doi:10.1007/978-1-4757-3837-7

[44] Paul R. Halmos. 1956. Homogeneous Locally Finite Polyadic Boolean Algebras of Infinite Degree. *Fundamenta Mathematicae* 43 (1956), 255–325.

[45] Paul R. Halmos and Steven Givant. 1998. *Logic as Algebra*. Dolciani Mathematical Expositions, Vol. 21. Mathematical Association of America.

[46] Makoto Hamana. 2006. An initial algebra approach to term rewriting systems with variable binders. *High. Order Symb. Comput.* 19, 2-3 (2006), 231–262. doi:10.1007/s10990-006-8747-5

[47] Makoto Hamana. 2011. Polymorphic Abstract Syntax via Grothendieck Construction. In *Foundations of Software Science and Computational Structures - 14th International Conference, FOSSACS 2011, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2011, Saarbrücken, Germany, March 26-April 3, 2011. Proceedings (Lecture Notes in Computer Science, Vol. 6604)*, Martin Hofmann (Ed.). Springer, 381–395. doi:10.1007/978-3-642-19805-2_26

[48] Robert Harper. 2016. *Practical Foundations for Programming Languages (2nd. Ed.)*. Cambridge University Press. https://www.cs.cmu.edu/%7Erwh/pfpl/index.html

[49] Leon Henkin, J. Donald Monk, and Alfred Tarski. 1971. *Cylindric Algebras. Part I*. Studies in Logic and the Foundations of Mathematics, Vol. 64. North-Holland.

[50] J.R. Hindley and J.P. Seldin. 2008. *Lambda-Calculus and Combinators: An Introduction*. Cambridge University Press.

[51] André Hirschowitz and Marco Maggesi. 2007. Modules over Monads and Linearity. In *Proc. of WoLLIC 2007 (Lecture Notes in Computer Science, Vol. 4576)*, Daniel Leivant and Ruy J. G. B. de Queiroz (Eds.). Springer, 218–237. doi:10.1007/978-3-540-73445-1_16

[52] André Hirschowitz and Marco Maggesi. 2010. Modules over monads and initial semantics. *Inf. Comput.* 208, 5 (2010), 545–564. doi:10.1016/j.ic.2009.07.003

[53] Martin Hofmann. 1999. Semantical Analysis of Higher-Order Abstract Syntax. In *14th Annual IEEE Symposium on Logic in Computer Science, Trento, Italy, July 2-5, 1999*. IEEE Computer Society, 204–213. doi:10.1109/LICS.1999.782616

[54] D.J. Howe. 1996. Proving Congruence of Bisimulation in Functional Programming Languages. *Inf. Comput.* 124, 2 (1996), 103–112.

[55] Gérard P. Huet. 1980. Confluent Reductions: Abstract Properties and Applications to Term Rewriting Systems: Abstract Properties and Applications to Term Rewriting Systems. *J. ACM* 27, 4 (1980), 797–821. doi:10.1145/322217.322230

[56] Atsushi Igarashi, Benjamin C. Pierce, and Philip Wadler. 1999. Featherweight Java: A Minimal Core Calculus for Java and GJ. In *Proc. of OOPSLA 1999*, Brent Hailpern, Linda M. Northrop, and A. Michael Berman (Eds.). ACM, 132–146. doi:10.1145/320384.320395

[57] André Joyal and Ross Street. 1991. The Geometry of Tensor Calculus, I. *Advances in Mathematics* 88, 1 (July 1991), 55–112. doi:10.1016/0001-8708(91)90003-P

[58] Gilles Kahn. 1987. Natural Semantics. In *Proceedings of the 4th Annual Symposium on Theoretical Aspects of Computer Science (STACS '87)*. Springer-Verlag, Berlin, Heidelberg, 22–39.

[59] G. A. Kavvos. 2019. Modalities, cohesion, and information flow. *Proc. ACM Program. Lang.* 3, POPL (2019), 20:1–20:29.

[60] Gregory M. Kelly. 2005. Basic Concepts of Enriched Category Theory. *Reprints in Theory and Applications of Categories* 10 (2005), 1–136.

[61] Stephen Cole Kleene. 1951. Representation of Events in Nerve Nets and Finite Automata. https://api.semanticscholar.org/CorpusID:60577779

[62] Jan Willem Klop. 1980. *Combinatory reduction systems.* Ph. D. Dissertation. Univ. Utrecht.

[63] Dexter Kozen. 1990. On Kleene Algebras and Closed Semirings. In *Proc. of MFCS 1990 (Lecture Notes in Computer Science, Vol. 452)*, Branislav Rovan (Ed.). Springer, 26–47. doi:10.1007/BFb0029594

[64] Dexter Kozen. 1992. On Action Algebras. *DAIMI Report Series* 21, 381 (Jan. 1992). doi:10.7146/dpb.v21i381.6613

[65] Jean-Louis Krivine. 1993. *Lambda-Calculus, Types and Models.* Ellis Horwood, New York. https://dl.acm.org/doi/10.5555/167408

[66] Thomas Lamiaux and Benedikt Ahrens. 2024. An Introduction to Different Approaches to Initial Semantics. *CoRR* abs/2401.09366 (2024). arXiv:2401.09366 doi:10.48550/ARXIV.2401.09366

[67] S.B. Lassen. 1998. *Relational Reasoning about Functions and Nondeterminism.* Ph. D. Dissertation. Dept. of Computer Science, University of Aarhus.

[68] F.William Lawvere. 2007. Axiomatic cohesion. *Theory and Applications of Categories [electronic only]* 19 (2007), 41–49.

[69] P.B. Levy. 2006. Infinitary howe's method. *Electr. Notes Theor. Comput. Sci.* 164, 1 (2006), 85–104.

[70] P.B. Levy, J. Power, and H. Thielecke. 2003. Modelling Environments in Call-by-Value Programming Languages. *Inf. Comput.* 185, 2 (2003), 182–210.

[71] C. Lüth. 1998. *Categorical Term Rewriting: Monads and Modularity.* Ph. D. Dissertation. University of Edinburgh.

[72] C. Lüth and N. Ghani. 1997. Monads and Modular Term Rewriting. In *Category Theory in Computer Science CTCS'97 (Lecture Notes in Computer Science, Vol. 1290).* Springer Verlag, Santa Margherita, Italy, 69–86.

[73] S. MacLane. 1971. *Categories for the Working Mathematician.* Springer-Verlag.

[74] Roger D. Maddux. 1997. Relation Algebras. In *Relational Methods in Computer Science*, Chris Brink, Wolfram Kahl, and Gunther Schmidt (Eds.). Springer, 22–38. doi:10.1007/978-3-7091-6510-2_2

[75] Per Martin-Löf. 1982. Constructive Mathematics and Computer Programming. In *Logic, Methodology and Philosophy of Science VI*, L. Jonathan Cohen, Jerzy Łoś, Helmut Pfeiffer, and Klaus-Peter Podewski (Eds.). Studies in Logic and the Foundations of Mathematics, Vol. 104. Elsevier, 153–175. doi:10.1016/S0049-237X(09)70189-2

[76] Per Martin-Löf. 1984. *Intuitionistic Type Theory.* Bibliopolis.

[77] Ian A. Mason and Carolyn L. Talcott. 1991. Equivalence in Functional Languages with Effects. *J. Funct. Program.* 1, 3 (1991), 287–327.

[78] Ralph Matthes and Tarmo Uustalu. 2003. Substitution in Non-wellfounded Syntax with Variable Binding. *Electronic Notes in Theoretical Computer Science* 82, 1 (2003), 191–205. doi:10.1016/S1571-0661(04)80639-X CMCS'03, Coalgebraic Methods in Computer Science (Satellite Event for ETAPS 2003).

[79] John McCarthy. 1962. Towards a Mathematical Science of Computation. In *Information Processing, Proceedings of the 2nd IFIP Congress 1962, Munich, Germany, August 27 - September 1, 1962.* North-Holland, 21–28.

[80] Robin Milner. 1989. *Communication and concurrency.* Prentice Hall.

[81] Robin Milner and Mads Tofte. 1991. Co-Induction in Relational Semantics. *Theor. Comput. Sci.* 87, 1 (1991), 209–220. doi:10.1016/0304-3975(91)90033-X

[82] Robin Milner, Mads Tofte, and Robert Harper. 1990. *Definition of standard ML.* MIT Press.

[83] Eugenio Moggi. 1989. Computational Lambda-Calculus and Monads. In *Proc. of LICS 1989.* IEEE Computer Society, 14–23.

[84] J. Morris. 1969. *Lambda Calculus Models of Programming Languages.* Ph. D. Dissertation. MIT.

[85] Mohammad Reza Mousavi, Michel A. Reniers, and Jan Friso Groote. 2007. SOS formats and meta-theory. *Theoretical Computer Science* 373, 3 (2007), 238–272.

[86] Lorenzo Pace. 2023. A Relation Algebra for Term Rewriting: A differential approach to sequential reduction (Revised Version). *CoRR* abs/2312.02996 (2023). doi:10.48550/ARXIV.2312.02996

[87] David Park. 1981. Concurrency and automata on infinite sequences. In *Theoretical Computer Science.* Springer Berlin Heidelberg, Berlin, Heidelberg, 167–183.

[88] F. Pfenning. 2016. *Computation and Deduction.* Cambridge University Press. https://books.google.it/books?id=N6YkPQAACAAJ

[89] Benjamin C. Pierce. 2002. *Types and programming languages.* MIT Press.

[90] A.M. Pitts. 2011. Howe's Method for Higher-Order Languages. In *Advanced Topics in Bisimulation and Coinduction*, D. Sangiorgi and J. Rutten (Eds.). Cambridge Tracts in Theoretical Computer Science, Vol. 52. Cambridge University Press, 197–232.

[91] A.M. Pitts. 2013. *Nominal Sets: Names and Symmetry in Computer Science.* Cambridge University Press.

[92] A. M. Pitts. 1988. Applications of Sup-lattice Enriched Category Theory to Sheaf Theory. *Proc. London Math. Soc.* 57 (1988), 433–480.

[93] Andrew M. Pitts. 1994. *Some Notes on Inductive and Co-Inductive Techniques in the Semantics of Functional Programs.* Technical Report BRICS-NS-94-5. BRICS, Department of Computer Science, University of Aarhus. https://cs.au.dk/~gerth/MC/ vi+135 pp, draft version.

[94] A. M. Pitts. 1997. Operationally-Based Theories of Program Equivalence. In *Semantics and Logics of Computation*, P. Dybjer and A. M. Pitts (Eds.). Cambridge University Press, 241–298.

[95] Andrew M. Pitts. 2002. Equivariant Syntax and Semantics. In *Automata, Languages and Programming, 29th International Colloquium, ICALP 2002, Malaga, Spain, July 8-13, 2002, Proceedings (Lecture Notes in Computer Science, Vol. 2380)*, Peter Widmayer, Francisco Triguero Ruiz, Rafael Morales Bueno, Matthew Hennessy, Stephan J. Eidenbenz, and Ricardo Conejo (Eds.). Springer, 32–36. doi:10.1007/3-540-45465-9_3

[96] Andrew M. Pitts. 2003. Nominal logic, a first order theory of names and binding. *Inf. Comput.* 186, 2, 165–193. doi:10.1016/S0890-5401(03)00138-X

[97] Andrew M. Pitts. 2006. Alpha-structural recursion and induction. *J. ACM* 53, 3 (2006), 459–506. doi:10.1145/1147954.1147961

[98] G.D. Plotkin. 1975. Call-by-name, call-by-value and the lambda-calculus. *Theoretical Computer Science* 1, 2 (1975), 125 – 159.

[99] Gordon D. Plotkin. 1981. *A Structural Approach to Operational Semantics.* Computer Science Department, Aarhus University.

[100] A.J. Power. 1989. An abstract formulation for rewrite systems. In *Category Theory and Computer Science*, D.H. Pitt, D.E. Rydehard, P. Dybjer, A.M. Pitts, and A. Poigné (Eds.), Vol. 389. Springer, 300–312.

[101] Vaughan R. Pratt. 1990. Action Logic and Pure Induction. In *Logics in AI, European Workshop, JELIA '90, Amsterdam, The Netherlands, September 10-14, 1990, Proceedings (Lecture Notes in Computer Science, Vol. 478)*, Jan van Eijck (Ed.). Springer, 97–120. doi:10.1007/BFb0018436

[102] Vaughan R. Pratt. 1992. Origins of the Calculus of Binary Relations. In *Proceedings of the Seventh Annual Symposium on Logic in Computer Science (LICS '92), Santa Cruz, California, USA, June 22-25, 1992.* IEEE Computer Society, 248–254. doi:10.1109/LICS.1992.185537

[103] Dag Prawitz. 1965. *Natural Deduction: A Proof-Theoretical Study.* Dover Publications, Mineola, N.Y.

[104] K.I. Rosenthal. 1990. *Quantales and their applications.* Longman Scientific & Technical.

[105] Gunther Schmidt. 2011. *Relational Mathematics.* Encyclopedia of Mathematics and its Applications, Vol. 132. Cambridge University Press.

[106] R. A. G. Seely. 1987. Modelling Computations: A 2-Categorical Framework. In *Proceedings of the Second Annual Symposium on Logic in Computer Science.* 65–71.

[107] P. Selinger. 2011. *A Survey of Graphical Languages for Monoidal Categories.* Springer Berlin Heidelberg, Berlin, Heidelberg, 289–355. doi:10.1007/978-3-642-12821-9_4

[108] M.H.B. Sørensen and P. Urzyczyn. 1998. *Lectures on the Curry-Howard Isomorphism.* Datalogisk Institut, Københavns Universitet. https://books.google.it/books?id=mm73GQAACAAJ

[109] Allen Stoughton. 1988. Substitution Revisited. *Theoretical Computer Science* 59, 3 (1988), 317–325. doi:10.1016/0304-3975(88)90149-1

[110] Georg Struth. 2001. Calculating Church-Rosser Proofs in Kleene Algebra. In *Proc. of RelMICS 2001 (Lecture Notes in Computer Science, Vol. 2561)*, Harrie C. M. de Swart (Ed.). Springer, 276–290. doi:10.1007/3-540-36280-0_19

[111] Georg Struth. 2006. Abstract abstract reduction. *J. Log. Algebraic Methods Program.* 66, 2 (2006), 239–270. doi:10.1016/j.jlap.2005.04.001

[112] Isar Stubbe. 2014. An introduction to quantaloid-enriched categories. *Fuzzy Sets Syst.* 256 (2014), 95–116. doi:10.1016/j.fss.2013.08.009

[113] Masako Takahashi. 1995. Parallel reductions in lambda-calculus. *Inf. Comput.* 118, 1 (1995), 120–127.

[114] Alfred Tarski. 1941. On the Calculus of Relations. *J. Symb. Log.* 6, 3 (1941), 73–89. doi:10.2307/2268577

[115] A. Tarski and S.R. Givant. 1988. *A Formalization of Set Theory Without Variables.* Number v. 41 in A formalization of set theory without variables. American Mathematical Soc.

[116] A. S. Troelstra and H. Schwichtenberg. 1996. *Basic Proof Theory.* Cambridge Tracts in Theoretical Computer Science, Vol. 43. Cambridge University Press, Cambridge.

[117] Daniele Turi and Gordon D. Plotkin. 1997. Towards a Mathematical Operational Semantics. In *Proceedings, 12th Annual IEEE Symposium on Logic in Computer Science, Warsaw, Poland, June 29 - July 2, 1997.* IEEE Computer Society, 280–291. doi:10.1109/LICS.1997.614955

[118] W. Wechler. 2012. *Universal Algebra for Computer Scientists.* Springer Berlin Heidelberg.

Appendix: Further Examples

Typed and Multi-Sorted Syntax

Multi-sorted syntax [38, 118] constraints term formation using a collection of *sorts*, or *types*, which are typically syntactically defined

themselves. For simplicity, we just consider a collection $\mathbb{S}$ of sorts and refine the notions of arity introduced so far accordingly. In the first-order case, arities become elements of[11] $\mathbb{S}^* \times \mathbb{S}$, and we write $\mathbf{op} : \mathsf{T}_1 \times \cdots \times \mathsf{T}_n \to \mathsf{T}$ when $|\mathbf{op}| = ((\mathsf{T}_1, \ldots, \mathsf{T}_n), \mathsf{T})$. In the second-order case, arities are elements of $(\mathbb{S}^* \times \mathbb{S})^* \times \mathbb{S}$, and we write $\mathbf{op} : [\vec{\mathsf{T}}_1]\mathsf{S}_1 \times \cdots \times [\vec{\mathsf{T}}_n]\mathsf{S}_n \to \mathsf{S}$ when $|\mathbf{op}| = ((\vec{\mathsf{T}}_1, \mathsf{S}_1), \cdots, (\vec{\mathsf{T}}_n, \mathsf{S}_n)), \mathsf{S})$. We abbreviate $[\,\cdot\,]\mathsf{T}$ as T. We also parametrise variables on sorts, so that we have mutually-disjoint collections $\mathcal{V}_\mathsf{T}$ for each sort T.

We extend the definition of second-order raw terms thus (first-order terms are defined similarly). A sorted context Γ is a finite sets of elements from $\bigcup_\mathsf{T} \mathcal{V}_\mathsf{T}$. We write $(\mathsf{x} : \mathsf{T}) \in \Gamma$ to indicate that $\mathsf{x} \in \mathcal{V}_\mathsf{T}$ and $\mathsf{x} \in \Gamma$. The sort-indexed family of raw terms $\mathcal{T}_{\Sigma,\mathsf{T}}(\mathcal{V})$ is defined using judgements of the form $\Gamma \vdash \mathsf{t} : \mathsf{T}$ and the following rules:

$$\frac{(\mathsf{x} : \mathsf{T}) \in \Gamma}{\Gamma \vdash \mathbf{var}\ \mathsf{x} : \mathsf{T}}$$

$$\frac{\Gamma, \vec{\mathsf{x}}_1 : \vec{\mathsf{S}}_1 \vdash \mathsf{t}_1 : \mathsf{T}_1 \quad \cdots \quad \Gamma, \vec{\mathsf{x}}_n : \vec{\mathsf{S}}_n \vdash \mathsf{t}_n : \mathsf{T}_n \quad \mathbf{op} : [\vec{\mathsf{S}}_1]\mathsf{T}_1 \times \cdots \times [\vec{\mathsf{S}}_n]\mathsf{T}_n \to \mathsf{T}}{\Gamma \vdash \mathbf{op}((\vec{\mathsf{x}}_1).\mathsf{t}_1, \ldots, (\vec{\mathsf{x}}_n).\mathsf{t}_n) : \mathsf{T}}$$

Finally, a term relation is a sort-indexed term relation $\phi(\mathsf{T})$ on $\mathcal{T}_{\Sigma,\mathsf{T}}(\mathcal{V})$. The structure of a SRA is defined pointwise. The same holds when taking the $\sim_\alpha$-quotient of $\mathcal{T}_{\Sigma,\mathsf{T}}(\mathcal{V})$.

Example .1 (System T)**.** Terms of System T [37] are defined as ordinary terms over the binding signature

$$\begin{aligned}\Sigma \triangleq \{ & \\ & \mathtt{lam} : (1), \mathtt{app} : (0,0), \mathtt{z} :: (), \\ & \mathtt{succ} : (0), \mathtt{natrec} : (0,0,2)\}\end{aligned}$$

The corresponding sorted signature is defined by taking sorts

$$\mathbb{S} \triangleq \{\mathsf{T} ::= \mathtt{nat} \mid \mathsf{T} \Rightarrow \mathsf{S}\}$$

and refining arities thus:

$$\begin{aligned}\mathtt{lam} &: [\mathsf{T}]\mathsf{S} \to \mathsf{T} \Rightarrow \mathsf{S} \\ \mathtt{app} &: (\mathsf{T} \Rightarrow \mathsf{S}) \times \mathsf{T} \to \mathsf{S} \\ \mathtt{z} &: \cdot \to \mathtt{nat} \\ \mathtt{succ} &: \mathtt{nat} \to \mathtt{nat} \\ \mathtt{natrec} &: \mathtt{nat} \times \mathsf{T} \times [\mathtt{nat}, \mathsf{T}]\mathsf{T} \to \mathsf{T}\end{aligned}$$

Syntax relations on the multi-sorted syntax indeed give ordinary term relations on second-order syntax restricted so as to respect types, meaning that they relate only terms having the same type — notation $\Gamma \vdash \mathsf{t}\ \phi\ \mathsf{s} : \mathsf{T}$. For example, we have the following operations:

$$\frac{(\mathsf{x} : \mathsf{T}) \in \Gamma}{\Gamma \vdash \mathsf{x}\ \Delta_\eta\ \mathsf{x} : \mathsf{T}} \qquad \frac{\Gamma, \mathsf{x} : \mathsf{T} \vdash \mathsf{t}\ \phi\ \mathsf{s} : \mathsf{S}}{\Gamma \vdash \lambda\mathsf{x}.\mathsf{t}\ \widetilde{\phi}\ \lambda\mathsf{x}.\mathsf{s} : \mathsf{T} \to \mathsf{S}}$$

$$\frac{\Gamma \vdash \mathsf{t}\ \phi\ \mathsf{t}' : \mathsf{T} \to \mathsf{S} \quad \Gamma \vdash \mathsf{s}\ \phi\ \mathsf{s}' : \mathsf{T}}{\Gamma \vdash \mathsf{ts}\ \widetilde{\phi}\ \mathsf{t}'\mathsf{s}' : \mathsf{S}}$$

$$\frac{}{\Gamma \vdash \mathsf{z}\ \widetilde{\phi}\ \mathsf{z} : \mathtt{nat}}$$

$$\frac{\Gamma \vdash \mathsf{t}\ \phi\ \mathsf{s} : \mathtt{nat}}{\Gamma \vdash \mathtt{succ}(\mathsf{t})\ \widetilde{\phi}\ \mathtt{succ}(\mathsf{s}) : \mathtt{nat}}$$

$$\frac{\Gamma \vdash \mathsf{t}\ \phi\ \mathsf{t}' : \mathtt{nat} \quad \Gamma \vdash \mathsf{s}\ \phi\ \mathsf{s}' : \mathsf{T} \quad \Gamma, \mathsf{x} : \mathtt{nat}, \mathsf{y} : \mathsf{T} \vdash \mathsf{u}\ \phi\ \mathsf{u}' : \mathsf{T}}{\Gamma \vdash \mathtt{natrec}(\mathsf{t}, \mathsf{s}, (\mathsf{x}, \mathsf{y}).\mathsf{u})\ \widetilde{\phi}\ \mathtt{natrec}(\mathsf{t}', \mathsf{s}', (\mathsf{x}, \mathsf{y}).\mathsf{u}) : \mathsf{T}}$$

We can define a notion of β-reduction as in previous examples. Notice, however, that to ensure that this relation is indeed a term relation, we need to prove a 'ground' preservation theorem: $\Gamma \vdash \mathsf{t} : \mathsf{T}$ and $\mathsf{t}\ \beta\ \mathsf{s}$ implies $\Gamma \vdash \mathsf{s} : \mathsf{T}$. This actually suggests to view typing itself as a term relation $\triangleright$ on the mono-sorted language obtained by extending (untyped) System T with the syntax of types (for instance, we shall define $\lambda\mathsf{x}.\mathsf{t} \triangleright \mathsf{S} \Rightarrow \mathsf{T}[\mathsf{S}/\mathsf{x}]$, provided $\mathsf{t} \triangleright \mathsf{T}$). That will allow us to express preservation as (we write $\triangleleft$ for $\triangleright^\circ$) $\triangleleft\ ;\ \beta \subseteq \triangleleft$ (and, perhaps, to infer $\triangleleft\ ;\ \beta^\mathsf{C} \subseteq \triangleleft$ from that). We leave this direction for future work.

Example .2 (Recursive Types)**.** Recursive types are defined as α-equivalence classes of closed raw types $\mathsf{T} ::= \mathtt{a} \mid \mathsf{T} \Rightarrow \mathsf{T} \mid \mu\mathtt{a}.\mathsf{T}$, where $\mu\mathtt{a}.\mathsf{T}$ binds $\mathtt{a}$ in T. The sorted binding signature of the λ-calculus with recursive types is obtained by taking (we have already defined the sorted arity of lambda abstraction and application) $\mathtt{fold} : \mathsf{T}[\mu\mathtt{a}.\mathsf{T}/\mathtt{a}] \to \mu\mathtt{a}.\mathsf{T}$ and $\mathtt{unfold} : \mu\mathtt{a}.\mathsf{T} \to \mathsf{T}[\mu\mathtt{a}.\mathsf{T}/\mathtt{a}]$. We extend the previously defined relation β with the term relation μ defined by $\Gamma \vdash \mathtt{unfold}(\mathtt{fold}(\mathsf{t}))\ \mu\ \mathsf{t} : \mathsf{T}[\mu\mathtt{a}.\mathsf{T}/\mathsf{T}]$.

Monoidal Syntax. A *monoidal signature* is a $(\mathbb{N} \times \mathbb{N})$-sorted signature containing operations $\mathtt{comp} : (n,m) \times (m,p) \to (n,p)$, $\mathtt{par} : (n,m) \times (n',m') \to (n+n', m+m')$, $\mathtt{sym}_{m,n} : (m+n, n+m)$, and $\mathtt{id}_n : (n,n)$. Given a monoidal signature Σ, the collection of *raw diagrams* is the multi-sorted set $\bigcup_{m,n} \mathcal{T}_{\Sigma,m,n}(\emptyset)$, and a term relation on them is a sorted relation $\phi \in \prod_{(m,n)} \wp(\mathcal{T}_{\Sigma,m,n}(\emptyset) \times \mathcal{T}_{\Sigma,m,n}(\emptyset))$. By previous considerations, we obtain a SRA structure, with Δ_η defined as the empty relation (diagrams do not involve variables). *String diagrams* are obtained by quotienting raw diagrams under the (term) relation induced by monoidal equivalence $\equiv$, such as $\mathtt{par}(\mathtt{id}_m, \mathtt{id}_n) \equiv \mathtt{id}_{m+n}$ and $\mathtt{par}(\mathtt{comp}(c,d), \mathtt{comp}(c',d')) \equiv \mathtt{comp}(\mathtt{par}(c,c'), \mathtt{par}($ (see [107]). Straightforward calculations show that the (complete) SRA structure on raw diagrams extends to string diagrams. Notice that term relations of the form ϕ^C correspond to string diagrams rewrite rules [12–14].

Reduced Syntax: Example.

Example .3. We refine Example .2 with $\mathtt{app} : \underline{(\mathsf{T} \Rightarrow \mathsf{S})} \times \mathsf{T} \to \mathsf{S}$ along with $\mathtt{unfold} : \underline{(\mu\mathtt{a}.\mathsf{T})} \to \mathsf{T}[\mu\mathtt{a}.\mathsf{T}/\mathtt{a}]\}$ as destructors, with $\mathtt{app} \sim \mathtt{lam}$ and $\mathtt{unfold} \sim \mathtt{fold}$. It is easy to see that taking relations on this language indeed gives a SRA. The reduction $\beta \cup \mu$ then satisfies both GIP and SGIP, due to typing.

The call-by-value counterpart of the language in Example .3 is defined by specifying a syntactic categories of *values*, whereby a term is a value if it is a λ-abstraction or a form $\mathtt{fold}(\mathsf{t})$ with t itself a value, and modifying reduction thus, where v ranges over values: $\Gamma \vdash \mathtt{app}(\mathtt{lam}(\mathsf{x}.\mathsf{t}), \mathsf{v})\ \beta\ \mathsf{t}[\mathsf{v}/\mathsf{x}] : \mathsf{T}$ and $\Gamma \vdash \mathtt{unfold}(\mathtt{fold}(\mathsf{v}))\ \mu\ \mathsf{v} : \mathsf{T}[\mu\mathtt{a}.\mathsf{T}/\mathtt{a}]$.

Identifying canonical terms and value, as one should indeed do, we notice that the constructor $\mathtt{fold}$ alone is now not enough to determine whether a term is a canonical: $\mathtt{fold}(\mathsf{t})$ is canonical only if t is. This not only breaks disjointness of constructors and destructors, but also makes the identification between canonical terms and introduction forms unsound.

A handy way to bypass this problem (besides modifying term relation algebras so as to distinguish between canonical terms and

[11] For a set A, we write A^* for $\bigcup_{n \geq 0} A^n$.

introduction forms) is to put calculi in so-called *reduced syntax* [67, 70, 83], which is a special case of *ordered-sorted syntax* [**?**]. In a nutshell, one consider (at least) two sorts, say val and term, ordered by val $\subseteq$ term. This means, in particular, that any term of sort val is also of sort term. A language is in reduced syntax if all its constructors have sort $\mathsf{T}_1 \times \cdots \times \mathsf{T}_n \to \mathsf{val}$, so that the constructor alone ensures that the whole term is a value (hence canonical). In our example, that means restricting the argument of `fold` to values. Because sorts are ordered, we can identify terms with expressions of sort term hence working in a mono-sorted language. The passage through val, however, ensures that introduction forms coincide with canonical terms.

Applying this methodology to Example .3 (it is a straightforward but tedious exercise to refine the general notions of syntax introduced so far in reduced form), we essentially obtain the usual grammatical categories of values (`v ::= lam(x.t) | fold(v)`) and terms (`t ::= x | v | app(t,t) | unfold(t)`). Term relations are then defined in the usual way on terms, as so are the operation $\overline{\cdot}$ and $\langle \cdot, \cdot \rangle$. Notice that we simulate `fold(t)` as `(app(lam(x.fold(x)), t)`.

On Proofs

All the proofs are given in calculational style [43]. With the exception of the existence of similarity, all results are proved on the finitary fragment of quantales, *viz.* action algebras

For ease of exposition, it is convenient to label the very definition of compatible refinement as a law itself. Notice that (S15) and **??** immediately give law (S23) in Figure 4 (*i.e.*$\widetilde{\Delta} \leq \Delta$).

(S15) $\quad \widehat{a} \triangleq \Delta_\eta \vee \widetilde{a}$

Moreover, we often compress straightforward deductive steps on pure relations as a single inference labelled with the generic tag (RA). It is also useful to spell out the following immediate derived equalities:

(S16) $\quad \Delta_\eta \,;\, \Delta_\eta = \Delta_\eta$

(S17) $\quad \Delta_\eta^\circ = \Delta_\eta$

(S18) $\quad \widetilde{a} \,;\, \Delta_\eta = \bot$

Proposition .4. *The laws in Figure 4 hold.*

Proof. Laws from (S19) to (S22) directly follows from the general theory of residuals, as well as (S33). We prove some the remaining laws by means of standard calculations.

(1) For law (S24) we first prove $\Delta[\Delta] \leq \Delta$. We have:

$$\begin{aligned}
&\Delta[\Delta] \leq \Delta \\
&\iff \Delta \leq \Delta \gg \Delta && (\sigma 6)\\
&\impliedby \widetilde{\Delta \gg \Delta} \leq \Delta \gg \Delta && \text{??}\\
&\iff \Delta_\eta \vee \widetilde{\Delta \gg \Delta} \leq \Delta \gg \Delta && (\text{S15})\\
&\iff (\Delta_\eta \vee \widetilde{\Delta \gg \Delta})[\Delta] \leq \Delta && (\sigma 6)\\
&\iff \Delta_\eta[\Delta] \vee (\widetilde{\Delta \gg \Delta})[\Delta] \leq \Delta && (\text{S22})\\
&\iff \Delta \vee (\widetilde{\Delta \gg \Delta})[\Delta] \leq \Delta && (\sigma 3)\\
&\impliedby (\widetilde{\Delta \gg \Delta})[\Delta] \leq \Delta && (\text{RA})\\
&\impliedby (\widetilde{\Delta \gg \Delta)[\Delta]} \leq \Delta && (\text{S28})\\
&\impliedby \widetilde{\Delta} \leq \Delta && (\text{S19})\\
&\impliedby (S23)
\end{aligned}$$

To see that $\Delta \leq \Delta[\Delta]$ holds too, notice that $\Delta = \Delta_\eta[\Delta] \leq \Delta[\Delta]$

(2) We prove (S25).

$$\begin{aligned}
&\widehat{a} \,;\, \widehat{b}\\
&= (\Delta_\eta \vee \widetilde{a}) \,;\, (\Delta_\eta \vee \widetilde{a}) && (\text{S15})\\
&= \Delta_\eta \,;\, \Delta_\eta \vee \Delta_\eta \,;\, \widetilde{b} \vee \widetilde{a} \,;\, \Delta_\eta \vee \widetilde{a} \,;\, \widetilde{b} && \text{RA}\\
&= \Delta_\eta \,;\, \Delta_\eta \vee \bot \vee \bot \vee \widetilde{a} \,;\, \widetilde{b} && \text{?? and (S18)}\\
&= \Delta_\eta \,;\, \Delta_\eta \vee \widetilde{a} \,;\, \widetilde{b} && \text{RA}\\
&= \Delta_\eta \vee \widetilde{a} \,;\, \widetilde{b} && (\text{S16})\\
&= \Delta_\eta \vee \widetilde{a \,;\, b} && \text{??}\\
&= \widehat{a \,;\, b} && (\text{S15})
\end{aligned}$$

(3) We prove (S26).

$$\begin{aligned}
&\widehat{a}^\circ\\
&= (\Delta_\eta \vee \widetilde{a})^\circ && (\text{S15})\\
&= \Delta_\eta^\circ \vee \widetilde{a}^\circ && \text{RA}\\
&= \Delta_\eta \vee \widetilde{a^\circ} && (\text{S17}) \text{ and } \text{??}\\
&= \widehat{a^\circ} && (\text{S15})
\end{aligned}$$

(4) We prove (S27).

$$\begin{aligned}
&\widehat{a} \,;\, b\\
&= (\Delta_\eta \vee \widetilde{a}) \,;\, b && (\text{S15})\\
&= \Delta_\eta \,;\, b \vee \widetilde{a} \,;\, b && (\text{S22})\\
&= b \vee \widetilde{a} \,;\, b; && (\sigma 3)\\
&\leq b \vee \widetilde{a \,;\, b} && (\sigma 4)\\
&\leq b \vee \Delta_\eta \vee \widetilde{a \,;\, b} && \text{RA}\\
&= b \vee \widehat{a \,;\, b} && (\text{S15})
\end{aligned}$$

□

Recall that a reduction is a term relation b such that

(B0) $\quad \Delta_\eta \,;\, b_0 = \bot$

Lemma .5. *$a^I \triangleq a[\Delta]$ is always CUI, for any relation a.*

$(\sigma 1)$ $(a\,;a')[b\,;b'] \le a[b]\,;a'[b']$
$(\sigma 2)$ $(a[b])^\circ = a^\circ[b^\circ]$
$(\sigma 3)$ $\Delta_\eta[a] = a = a[\Delta_\eta]$
$(\sigma 4)$ $(a[b])[c] = a[b[c]]$
$(\sigma 4)$ $\widetilde{a}[b] \le \widetilde{a[b]}$
$(\sigma 5)$ $a \le a' \;\&\; b \le b' \implies a[b] \le a'[b']$
$(\sigma 6)$ $a[b] \le c \iff a \le b \gg c$

Figure 1: Basic Laws

$(\zeta 1)$ $\Box(a\,;b) = \Box a\,;\Box b$
$(\zeta 2)$ $(\Box a)^\circ = \Box a^\circ$
$(\zeta 3)$ $a \le b \implies \Box a \le \Box b$
$(\zeta 4)$ $\Box a \le a$
$(\zeta 5)$ $\Box a \le \Box\Box a$
$(\zeta 6)$ $\Box\Delta\,;a\,;\Box\Delta \le \Box a$
$(\zeta 7)$ $\Box\Delta_\eta \le \bot$
$(\zeta 8)$ $(\Box a)[b] = \Box a$
$(\zeta 9)$ $\Box a \le b \iff a \le \blacklozenge b$

Figure 2: Basic Laws

$(\varphi 1)$ $\overline{a\,;b} = \overline{a}\,;\overline{b}$
$(\varphi 2)$ $\overline{a}^\circ = \overline{a^\circ}$
$(\varphi 3)$ $\overline{a}[b] \le \overline{a[b]}$
$(\varphi 4)$ $a \le b \implies \overline{a} \le \overline{b}$
$(\varphi 5)$ $\langle a\,;a', b\,;b'\rangle = \langle a, b\rangle\,;\langle a', b'\rangle$
$(\varphi 6)$ $\langle a, b\rangle^\circ = \langle a^\circ, b^\circ\rangle$
$(\varphi 7)$ $\langle a, b\rangle[c] \le \langle a[c], b[c]\rangle$
$(\varphi 8)$ $a_i \le b_i \implies \langle a_1, a_2\rangle \le \langle b_1, b_2\rangle$
$(\varphi 9)$ $\widetilde{a} = \overline{a} \vee \langle a, a\rangle$
$(\varphi 10)$ $\overline{a}\,;\langle b, b\rangle \le \bot$
$(\varphi 11)$ $\Box\langle a, b\rangle = \langle \Box a, b\rangle$

Figure 3: Basic Laws

(S19) $(a \gg b)[a] \le b$
(S20) $a \le (b \gg a[b])$
(S21) $(a \gg b)\,;(a' \gg b') \le (a\,;a') \gg (b\,;b')$
(S22) $(a \vee b)[c] = a[c] \vee b[c]$
(S23) $\widetilde{\Delta} \le \Delta$
(S24) $\Delta[\Delta] = \Delta$
(S25) $\widehat{a\,;b} = \widehat{a}\,;\widehat{b}$
(S26) $\widehat{a^\circ} = \widehat{a}^\circ$
(S27) $\widehat{a}[b] \le \widehat{a[b]} \vee b$
(S28) $\widetilde{a \gg b} \le a \gg \widetilde{b}$
(S29) $\widehat{a \gg b} \le a \gg \widehat{b} \vee a$
(S33) $a' \le a \;\&\; b \le b' \implies a \gg b \le a' \gg b'$
(S34) $a \le b \implies \widehat{a} \le \widehat{b}$

Figure 4: Derived Laws

Proof. Calculate:

$$\begin{aligned} & a^{\mathrm{II}} \\ &= (a[1])[1] && (??)\\ &= a[1[1]] && (\sigma 4)\\ &= a[1] && (\mathrm{S24})\\ &= a^{\mathrm{I}} && (??) \end{aligned}$$

□

Proposition .6. *The following laws are derivable.*
(S36) $\widehat{a} \le a \implies \Delta[a] \le a$
(S37) $a[a] \le a \implies \widetilde{a}[a] \le \widetilde{a}$

(H1) $a \le a^{\mathrm{C}}$
(H2) $a^{\mathrm{CC}} = a^{\mathrm{C}}$
(H3) $a^{\mathrm{C}} = a \vee \widehat{a^{\mathrm{C}}}$
(H4) $a^{\mathrm{H}} = \widehat{a^{\mathrm{H}}}\,;a$
(H5) $a \le b \implies a^{\mathrm{C}} \le b^{\mathrm{C}}$
(H6) $a \vee \widehat{b} \le b \implies a^{\mathrm{C}} \le b$
(H7) $\widehat{b}\,;a \le b \implies a^{\mathrm{H}} \le b$

Figure 5: Howe Extension

(S38) $a[a] = a \implies \widetilde{a}[a] = \widetilde{a}$

Proof. We calculate thus: For (S36), we have:

$$\begin{aligned} & \Delta[a] \le a \\ &\iff \Delta \le a \gg a && (\sigma 6)\\ &\impliedby \widehat{a \gg a} \le a \gg a && ??\\ &\impliedby a \gg \widehat{a} \vee a \le a \gg a && (\mathrm{S29})\\ &\impliedby \widehat{a} \le a && (\mathrm{S33}) \end{aligned}$$

For (S37), we have:

$$\begin{aligned} & \widetilde{a}[a] \le \widetilde{a} \\ &\impliedby a\widetilde{[a]} \le \widetilde{a} && (\sigma 4)\\ &\impliedby a[a] \le a && ?? \end{aligned}$$

□

Proposition .7. *The laws in Figure 5 holds in any (some are actually axioms, which we report for ease of readability).*

(H8) $\Delta \leq a^{\mathbf{C}}$

(H9) $\Delta^{\mathbf{C}} = \Delta$

(H10) $a^{\mathbf{C}\circ} \leq a^{\circ\mathbf{C}}$

(H11) $(a\,;b)^{\mathbf{C}} \leq a^{\mathbf{C}}\,;b^{\mathbf{C}}$

(H12) $a^{*\mathbf{C}} \leq a^{\mathbf{C}*}$

(H13) $\Delta[a^{\mathbf{C}}] \leq a^{\mathbf{C}}$

(H14) $a \leq \Delta \implies a \leq a^{\mathbf{H}}$

(H15) $a[\Delta] \leq a \implies a^{\mathbf{H}}[a^{\mathbf{H}}] \leq a^{\mathbf{H}}$

(H16) $\Delta \leq a \implies \widehat{a} \leq a$

(H17) $a\,;a \leq a \implies a^{\mathbf{H}}\,;a \leq a^{\mathbf{H}}$

Figure 6: Operators $\cdot^{\mathbf{C}}$ and $\cdot^{\mathbf{H}}$: Derived Laws

Notice that $a^{\mathbf{H}}$ gives the least solution to the a-parametric equation

$$x = \widehat{x}\,;a \qquad \dagger$$

Such a solution not only exists, but it is also unique.

Proposition .8. *Equation † has a unique solution. In particular, $a^{\mathbf{H}}$ is a solution; moreover, if b, c are solutions of †, we have $b = c$. That is:*

$$b = \widehat{b}\,;a \;\&\; c = \widehat{c}\,;a \implies b = c.$$

Proof. We prove that whenever

$$b = \widehat{b};a \qquad \dagger_1$$
$$c = \widehat{c};a \qquad \dagger_2$$

we have $b = c$. Notice that it is sufficient to show $b \leq c$ (the other direction follows by renaming). We calculate:

$$\begin{aligned}
& b \leq c \\
\iff\ & \Delta; b \leq c && \text{RA} \\
\iff\ & \Delta \leq c \leftarrow b && \text{RA} \\
\impliedby\ & \widehat{c \leftarrow b} \leq c \leftarrow b && \text{??} \\
\impliedby\ & \widehat{c \leftarrow b}; b \leq c && \text{RA} \\
\iff\ & \widehat{c \leftarrow b}; \widehat{b}; a \leq c && \dagger_1 \\
\iff\ & (\widehat{c \leftarrow b}); b; a \leq c && \text{(S25)} \\
\iff\ & \widehat{c}; a \leq c && \text{RA} \\
\impliedby\ & c \leq c && \dagger_2
\end{aligned}$$

□

A Evaluation Fragment

Proposition A.1. *The following laws are derivable.*

(K12) $a \leq \Diamond b \iff \Box(a[\Delta]) \leq b$

(K13) $\Box\Delta \leq \Delta$

(K14) $\Box(a \vee b) = \Box a \vee \Box b$

(K15) $\Box\blacklozenge a \leq a$

(K16) $a \leq \blacklozenge\Box a$

(K17) $a \leq \Delta \implies \Box a \leq \Delta$

(K18) $\Diamond a\,;\Diamond b \leq \Diamond(a\,;b)$

(K19) $(\Diamond a)^{\circ} \leq \Diamond a^{\circ}$

Proof. We prove law (K19) as an illustrative example. We calculate:

$$\begin{aligned}
& (\Diamond a)^{\circ} \leq \Diamond a^{\circ} \\
\iff\ & \Box(\Diamond a)^{\circ}[\Delta] \leq a^{\circ} && \text{(K12)} \\
\iff\ & \Box(\Diamond a)^{\circ}[\Delta^{\circ}] \leq a^{\circ} && \text{RA} \\
\iff\ & \Box((\Diamond a)[\Delta])^{\circ} \leq a^{\circ} && (\sigma 2) \\
\iff\ & (\Box(\Diamond a)[\Delta])^{\circ} \leq a^{\circ} && (\zeta 2) \\
\impliedby\ & \Box(\Diamond a)[\Delta] \leq a && \text{RA} \\
\iff\ & \Diamond a \leq \Diamond a && \text{(K12)}
\end{aligned}$$

□

Proposition A.2. *Let a be a* closed *relation.*[12] *Then, the following laws hold.*

(K19) $a = \Box\Diamond a$

(K20) $\Diamond\Box\Delta = \Delta$

(K21) $1 \leq a \iff \Delta \leq \Diamond a$

(K22) $a\,;a \leq a \implies \Diamond a\,;\Diamond a \leq \Diamond a$

(K23) $a^{\circ} \leq a \implies \Diamond a^{\circ} \leq \Diamond a$

Definition A.3. *A relation a* respects canonical terms *if:*

(RCN) $\overline{\Delta}\,;a \leq a\,;\overline{\Delta}$

A relation a respects complete terms *if:*

(RCP) $\Box\Delta\,;a \leq a\,;\Box\Delta$

Lemma A.4. *The following are derivable (in particular, $\widehat{a}$ always respects canonical terms, whereas evaluation refinement respects complete terms):*

(RC1) $\overline{\Delta}\,;\widehat{a} \leq \widehat{a}\,;\overline{\Delta}$

(RC2) $\Delta_{\kappa}\,;\Box\widehat{a} \leq \Box\widehat{a}\,;\Delta_{\kappa}$

(RC3) $\Box\Delta\,;\langle a, b\rangle \leq \langle a, b\rangle\,;\Box\Delta$

(RC4) $\Box\Delta\,;\langle a, b\rangle \leq \Box\langle a, b\rangle$

(RC5) $\Box c\,;\langle a, b\rangle \leq \Box(c\,;\langle a, b\rangle)$

Lemma A.5. *The following are derivable:*

(C19) $\widehat{a}\,;\langle b, c\rangle \leq \langle a, a\rangle\,;\langle b, c\rangle$

[12] The assumption on a is not necessary. We may in fact drop it and consider the relation $\Box a$, hence obtaining the laws:

$$\Box a = \Box\Diamond\Box a$$
$$\Diamond\Box\Delta = \Delta$$
$$\Box\Delta \leq \Box a \iff \Delta \leq \Diamond\Box a$$
$$\Box a\,;\Box a \leq \Box a \implies \Diamond\Box a\,;\Diamond\Box a \leq \Diamond\Box a$$
$$\Box a^{\circ} \leq \Box a \implies \Diamond\Box a^{\circ} \leq \Diamond\Box a$$

(F1) $\Delta_\kappa \leq a^{\Downarrow}$

(F2) $a \,;\, a^{\Downarrow} \leq a^{\Downarrow}$

(F3) $\langle a^{\Downarrow} \rangle \,;\, a^{\Downarrow} \leq a^{\Downarrow}$

(F4) $\Delta_\kappa \vee a \,;\, b \vee \langle b \rangle \,;\, b \leq b \implies a^{\Downarrow} \leq b$

Figure 7: Laws of $\cdot^{\Downarrow}$

(F5) $\Delta^{\Downarrow} \leq \Delta$

(F6) $a^{\Downarrow} = a^{\Downarrow} \,;\, \Delta_\kappa$

(F7) $a \leq b \implies a^{\Downarrow} \leq b^{\Downarrow}$

(F8) $a \leq \Box a \implies a^{\Downarrow} \leq \Box(a^{\Downarrow})$

(F9) $\Delta_\kappa \vee b \,;\, a^{\circ} \vee b \,;\, \langle b \rangle \leq b \implies a^{\Downarrow\circ} \leq b$

(F10) $\mathsf{Inv} \implies \Delta_\kappa \,;\, a^{\Downarrow} = \Delta_\kappa$

(F11) $\mathsf{Inv} \implies a^{\Downarrow} \,;\, a^{\Downarrow *} = a^{\Downarrow}$

Figure 8: Derived Laws of $a^{\Downarrow}$

A.1 Big-Step Evaluation

Lemma A.6. *If a is closed, then $a^{\Downarrow}$ is the least solution of:*

$$x = \Delta_\kappa \vee a \,;\, x \vee \langle x \rangle \,;\, x.$$

Definition A.7. *Closed inversion principle on $a = \Box a$:*

$$a = \langle \Delta_\kappa \rangle ; a \qquad \mathsf{Inv}$$

Remark A.8. From now on, we assume all rules of computation to be subject to Inv. Notice that this indeed ensures that rules of computation do not act on canonical expressions:

$$\Delta_\kappa \,;\, a \overset{\mathsf{Inv}}{=} \Delta_\kappa \,;\, \langle \Delta_\kappa \rangle \,;\, a \leq \overline{\Delta} \,;\, \langle \Delta_\kappa \rangle \,;\, a \overset{??}{=} \bot.$$

Proposition A.9 (Inversion Lemma)**.** *Suppose a is a closed relation satisfying Inv. Then,*

$$\langle b \rangle \,;\, a^{\Phi} \leq \langle b \,;\, a^{\Phi} \rangle \,;\, a^{\Phi} \qquad \text{Inversion}$$

Proposition A.10. *The laws in Figure 8 are derivable.*

Proof. Laws (F5) and (F7) are straightforward, whereas laws (F9) is obtained by dualising axiom (F4). Moreover, (F8) follow from Proposition **??**. To prove law (F6), it is actually enough to show $a^{\Downarrow} \leq a^{\Downarrow} \,;\, \Delta_\kappa$, since the other inclusion follows thus:

$$a^{\Downarrow} \,;\, \Delta_\kappa \leq a^{\Downarrow} \,;\, \Delta = a^{\Downarrow}.$$

Hence, we calculate:

$$\begin{aligned}
& a^{\Downarrow} \leq a^{\Downarrow} \,;\, \Delta_\kappa \\
& \quad \Longleftarrow \ \Delta_\kappa \vee a \,;\, a^{\Downarrow} \,;\, \Delta_\kappa \vee \langle a^{\Downarrow} \,;\, \Delta_\kappa \rangle \,;\, a^{\Downarrow} \,;\, \Delta_\kappa \leq a^{\Downarrow} \,;\, \Delta_\kappa && \text{(F4)} \\
& \quad \Longleftrightarrow \ \Delta_\kappa \,;\, \Delta_\kappa \vee a \,;\, a^{\Downarrow} \,;\, \Delta_\kappa \vee \langle a^{\Downarrow} \,;\, \Delta_\kappa \rangle \,;\, a^{\Downarrow} \,;\, \Delta_\kappa \leq a^{\Downarrow} \,;\, \Delta_\kappa \\
& \qquad\qquad (\Delta_\kappa \leq \Delta \text{ implies } \Delta_\kappa = \Delta_\kappa \,;\, \Delta_\kappa) \\
& \quad \Longleftarrow \ a^{\Downarrow} \,;\, \Delta_\kappa \vee a \,;\, a^{\Downarrow} \,;\, \Delta_\kappa \vee \langle a^{\Downarrow} \,;\, \Delta_\kappa \rangle \,;\, a^{\Downarrow} \,;\, \Delta_\kappa \leq a^{\Downarrow} \,;\, \Delta_\kappa && \text{(F1)} \\
& \quad \Longleftrightarrow \ a \,;\, a^{\Downarrow} \,;\, \Delta_\kappa \vee \langle a^{\Downarrow} \,;\, \Delta_\kappa \rangle \,;\, a^{\Downarrow} \,;\, \Delta_\kappa \leq a^{\Downarrow} \,;\, \Delta_\kappa && \text{RA} \\
& \quad \Longleftarrow \ a^{\Downarrow} \,;\, \Delta_\kappa \vee \langle a^{\Downarrow} \,;\, \Delta_\kappa \rangle \,;\, a^{\Downarrow} \,;\, \Delta_\kappa \leq a^{\Downarrow} \,;\, \Delta_\kappa && \text{(F2)} \\
& \quad \Longleftrightarrow \ \langle a^{\Downarrow} \,;\, \Delta_\kappa \rangle \,;\, a^{\Downarrow} \,;\, \Delta_\kappa \leq a^{\Downarrow} \,;\, \Delta_\kappa && \text{RA} \\
& \quad \Longleftarrow \ \langle a^{\Downarrow} \,;\, \Delta \rangle \,;\, a^{\Downarrow} \,;\, \Delta_\kappa \leq a^{\Downarrow} \,;\, \Delta_\kappa && (\Delta_\kappa \leq \Delta) \\
& \quad \Longleftrightarrow \ \langle a^{\Downarrow} \rangle \,;\, a^{\Downarrow} \,;\, \Delta_\kappa \leq a^{\Downarrow} \,;\, \Delta_\kappa && \text{RA} \\
& \quad \Longleftarrow \ a^{\Downarrow} \,;\, \Delta_\kappa \leq a^{\Downarrow} \,;\, \Delta_\kappa && \text{(F3)}
\end{aligned}$$

Let us now move to law (F10). Assume Inv. By (F1), it is enough to show $\Delta_\kappa \,;\, a^{\Downarrow} \leq \Delta_\kappa$.[13] We calculate:

$$\begin{aligned}
& \Delta_\kappa \,;\, a^{\Downarrow} \leq \Delta_\kappa \\
& \quad \Longleftrightarrow \ a^{\Downarrow} \leq \Delta_\kappa \to \Delta_\kappa && \text{RA} \\
& \quad \Longleftarrow \ \Delta_\kappa \vee a \,;\, (\Delta_\kappa \to \Delta_\kappa) \vee \langle \Delta_\kappa \to \Delta_\kappa \rangle \,;\, (\Delta_\kappa \to \Delta_\kappa) \leq \Delta_\kappa \to \Delta_\kappa \\
& \qquad\qquad \text{(F4)} \\
& \quad \Longleftrightarrow \ \Delta_\kappa \,;\, (\Delta_\kappa \vee a \,;\, (\Delta_\kappa \to \Delta_\kappa) \vee \langle \Delta_\kappa \to \Delta_\kappa \rangle \,;\, (\Delta_\kappa \to \Delta_\kappa)) \leq \Delta_\kappa \\
& \qquad\qquad \text{RA} \\
& \quad \Longleftrightarrow \ \Delta_\kappa \,;\, \Delta_\kappa \vee \Delta_\kappa \,;\, a \,;\, (\Delta_\kappa \to \Delta_\kappa) \vee \Delta_\kappa \,;\, \langle \Delta_\kappa \to \Delta_\kappa \rangle \,;\, (\Delta_\kappa \to \Delta_\kappa) \leq \Delta_\kappa \\
& \qquad\qquad \text{RA} \\
& \quad \Longleftrightarrow \ \Delta_\kappa \,;\, a \,;\, (\Delta_\kappa \to \Delta_\kappa) \vee \Delta_\kappa \,;\, \langle \Delta_\kappa \to \Delta_\kappa \rangle \,;\, (\Delta_\kappa \to \Delta_\kappa) \leq \Delta_\kappa \\
& \qquad\qquad \text{RA, } \Delta_\kappa = \Delta_\kappa \,;\, \Delta_\kappa \\
& \quad \Longleftarrow \ \bot \vee \Delta_\kappa \,;\, \langle \Delta_\kappa \to \Delta_\kappa \rangle \,;\, (\Delta_\kappa \to \Delta_\kappa) \leq \Delta_\kappa \\
& \qquad\qquad (\text{ Inv implies } \Delta_\kappa \,;\, a \leq \bot) \\
& \quad \Longleftrightarrow \ \Delta_\kappa \,;\, \langle \Delta_\kappa \to \Delta_\kappa \rangle \,;\, (\Delta_\kappa \to \Delta_\kappa) \leq \Delta_\kappa && \text{RA} \\
& \quad \Longleftarrow \ \bot \leq \Delta_\kappa && ?? \text{ and } \Delta_\kappa \leq \overline{\Delta}
\end{aligned}$$

Finally, for law (F11) we assumeInv and notice that it is enough to show $a^{\Downarrow} \,;\, a^{\Downarrow *} \leq a^{\Downarrow}$, since $a^{\Downarrow} = a^{\Downarrow} \,;\, \Delta \leq a^{\Downarrow} \,;\, a^{\Downarrow *}$. Hence, we calculate:

$$\begin{aligned}
& a^{\Downarrow} \,;\, a^{\Downarrow *} \leq a^{\Downarrow} \\
& \quad \Longleftarrow \ a^{\Downarrow} \vee a^{\Downarrow} \,;\, a^{\Downarrow} \leq a^{\Downarrow} && ?? \\
& \quad \Longleftarrow \ a^{\Downarrow} \,;\, a^{\Downarrow} \leq a^{\Downarrow} && \text{RA} \\
& \quad \Longleftrightarrow \ a^{\Downarrow} \,;\, \Delta_\kappa \,;\, a^{\Downarrow} \leq a^{\Downarrow} && \text{(F6)} \\
& \quad \Longleftrightarrow \ a^{\Downarrow} \,;\, \Delta_\kappa \leq a^{\Downarrow} && \text{(F10)} \\
& \quad \Longleftrightarrow \ a^{\Downarrow} \leq a^{\Downarrow} && \text{(F6)}
\end{aligned}$$

□

Lemma A.11. $\langle \Delta_\kappa \rangle \,;\, a^{\Downarrow} \leq a \,;\, a^{\Downarrow}$.

Theorem A.12 (Determinism)**.** *Let a be a* closed rule of computation. *Then: $a \,;\, a^{\circ} \leq \Delta \implies a^{\Downarrow\circ} \,;\, a^{\Downarrow} \leq \Delta_\kappa$*

Proof. We split the proof in two parts. First, we prove that under the assumption $a \,;\, a^{\circ} \leq \Delta$ we have $a^{\circ} \,;\, a^{\Downarrow} \leq a^{\Downarrow}$. We then show

[13] As for the previous proof, we have: $\Delta_\kappa \overset{RA}{=} \Delta_\kappa \,;\, \Delta_\kappa \overset{\text{(F1)}}{\leq} \Delta_\kappa \,;\, a^{\Downarrow}$.

that from the latter, the (in)equality $a^{\Downarrow\circ} \,;\, a^{\Downarrow} \leq \Delta_\kappa$ follows, hence concluding the desired thesis. To prove $a^\circ \,;\, a^{\Downarrow} \leq a^{\Downarrow}$, we calculate:

$$\begin{aligned}
& a^\circ \,;\, a^{\Downarrow} \\
&= a^\circ \,;\, \langle \Delta_\kappa \rangle \,;\, a^{\Downarrow} && \text{Inv} \\
&\leq a^\circ \,;\, a \,;\, a^{\Downarrow} && (\langle \Delta_\kappa \rangle \,;\, a^{\Downarrow} \leq a \,;\, a^{\Downarrow}) \\
&\leq a^{\Downarrow} && (a^\circ \,;\, a \leq \Delta)
\end{aligned}$$

Now for $a^{\Downarrow\circ} \,;\, a^{\Downarrow} \leq \Delta_\kappa$. We calculate:

$$\begin{aligned}
& a^{\Downarrow\circ} \,;\, a^{\Downarrow} \leq \Delta_\kappa \\
&\iff a^{\Downarrow\circ} \leq \Delta_\kappa \leftarrow a^{\Downarrow} && \text{RA} \\
&\impliedby \Delta_\kappa \vee (\Delta_\kappa \leftarrow a^{\Downarrow}) \,;\, a^\circ \vee (\Delta_\kappa \leftarrow a^{\Downarrow}) \,;\, \langle \Delta_\kappa \leftarrow a^{\Downarrow} \rangle \leq \Delta_\kappa \leftarrow a^{\Downarrow} && \text{(F9)} \\
&\iff \Delta_\kappa \,;\, a^{\Downarrow} \vee (\Delta_\kappa \leftarrow a^{\Downarrow}) \,;\, a^\circ \,;\, a^{\Downarrow} \vee (\Delta_\kappa \leftarrow a^{\Downarrow}) \,;\, \langle \Delta_\kappa \leftarrow a^{\Downarrow} \rangle \,;\, a^{\Downarrow} \leq \Delta_\kappa && \text{RA} \\
&\iff \Delta_\kappa \vee (\Delta_\kappa \leftarrow a^{\Downarrow}) \,;\, a^\circ \,;\, a^{\Downarrow} \vee (\Delta_\kappa \leftarrow a^{\Downarrow}) \,;\, \langle \Delta_\kappa \leftarrow a^{\Downarrow} \rangle \,;\, a^{\Downarrow} \leq \Delta_\kappa && \text{(F10)} \\
&\iff (\Delta_\kappa \leftarrow a^{\Downarrow}) \,;\, a^\circ \,;\, a^{\Downarrow} \vee (\Delta_\kappa \leftarrow a^{\Downarrow}) \,;\, \langle \Delta_\kappa \leftarrow a^{\Downarrow} \rangle \,;\, a^{\Downarrow} \leq \Delta_\kappa && \text{RA} \\
&\impliedby (\Delta_\kappa \leftarrow a^{\Downarrow}) \,;\, a^{\Downarrow} \vee (\Delta_\kappa \leftarrow a^{\Downarrow}) \,;\, \langle \Delta_\kappa \leftarrow a^{\Downarrow} \rangle \,;\, a^{\Downarrow} \leq \Delta_\kappa && a^\circ \,;\, a^{\Downarrow} \leq a^{\Downarrow} \\
&\impliedby (\Delta_\kappa \leftarrow a^{\Downarrow}) \,;\, \langle \Delta_\kappa \leftarrow a^{\Downarrow} \rangle \,;\, a^{\Downarrow} \leq \Delta_\kappa && \text{RA} \\
&\impliedby (\Delta_\kappa \leftarrow a^{\Downarrow}) \,;\, \langle (\Delta_\kappa \leftarrow a^{\Downarrow}) \,;\, a^{\Downarrow} \rangle \,;\, a^{\Downarrow} \leq \Delta_\kappa && \text{Inversion} \\
&\impliedby (\Delta_\kappa \leftarrow a^{\Downarrow}) \,;\, \langle \Delta_\kappa \rangle \,;\, a^{\Downarrow} \leq \Delta_\kappa && \text{RA} \\
&\impliedby (\Delta_\kappa \leftarrow a^{\Downarrow}) \,;\, a^{\Downarrow} \leq \Delta_\kappa && \text{RA and ??} \\
&\impliedby \Delta_\kappa \leq \Delta_\kappa && \text{RA} \\
& && \square
\end{aligned}$$

B Applicative Similarity

Let us fix a closed reduction b.

Definition B.1. *Define the* $B(x) \triangleq (b^E \,;\, \overline{x}) \leftarrow b^E$.

Notice that B is monotone and that $B(a)$ is always closed, *i.e.* $\square B(a) = B(a)$.

Definition B.2. *A* b-simulation *is any relation such that*

$$a \leq B(\Diamond a)$$

is derivable. Equivalently: $a \,;\, b^E \leq b^E \,;\, \overline{\Diamond a}$. *We say that* a *is a* b-bisimulation *if* $a \leq \Diamond a \wedge B(\Diamond a^\circ)^\circ$ *is derivable Since* b *is fixed, we just talk of "(bi)simulation".*

Call *similarity* and write sim for the greatest solution to $x = B(\Diamond x)$ (since b is fixed, we can treat similarity as a constant). We thus have the laws:

(B1) $\quad \mathsf{sim} = (b^{\mathbf{E}} \,;\, \overline{\Diamond \mathsf{sim}}) \leftarrow b^{\mathbf{E}}$

(B2) $\quad a \leq (b^{\mathbf{E}} \,;\, \overline{\Diamond a}) \leftarrow b^{\mathbf{E}} \implies a \leq \mathsf{sim}$

Proposition B.3. *Similarity is a closed preorder. That is:*

(B3) $\quad \square\Delta \leq \mathsf{sim}$

(B4) $\quad \mathsf{sim} \,;\, \mathsf{sim} \leq \mathsf{sim}$

(B5) $\quad \square\mathsf{sim} = \mathsf{sim}$

Proposition B.4. *The relation* $\Diamond\mathsf{sim}$ *is the greatest solution to the equation* $x = \Diamond B$. *Hence, we have:*

(B7) $\quad \Diamond\mathsf{sim} = \Diamond((b^E \,;\, \overline{\Diamond\mathsf{sim}}) \leftarrow b^E)$

(B8) $\quad a \leq \Diamond((b^E \,;\, \overline{\Diamond a}) \leftarrow b^E) \implies a \leq \Diamond\mathsf{sim}$

Definition B.5. *We say that* b *is* structural *if for any compatible relation* a, *we have* $\langle \widehat{a}, a \rangle \,;\, b \leq b \,;\, a[a]$. *Notice that this corresponds to the proof-theoretic semantic notion of* harmony.

We thus add to our theory the axiom:

(Harmony) $\quad \widehat{a} \leq a \implies \langle \widehat{a}, a \rangle \,;\, b \leq b \,;\, a[a]$

Howe's Method

We prove compatibility of $\Diamond\mathsf{sim}$ following *Howe's method* [54, 90], properly algebrised into ERAs.

Definition B.6. *We define the* open op-Howe extension *of a closed relation* a *as* $a^{\S} \triangleq (\Diamond a)^{H^\circ}$. *Unfolding the definition of the op-Howe operator, we see that* $a^{\S}$ *is least solution to the equation* $x = \Diamond a \,;\, \widehat{x}$.

Lemma B.7. *Say that a closed relation* a *is a closed preorder if it is transitive and* $\square\Delta \leq a$. *Then, for any closed preorder* a, *we have:*

(§1) $\quad \Delta \leq a^{\S}$

(§2) $\quad a^{\S}[a^{\S}] \leq a^{\S}$

(§3) $\quad a^{\S} \leq \Diamond a \,;\, \widehat{a^{\S}}$

(§4) $\quad \Diamond a \,;\, a^{\S} \leq a^{\S}$.

We are now going to prove that if a is a closed preorder and a simulation, then $a^{\S}$ is an open simulation. Since open similarity is the greatest such simulation, it follows that $\mathsf{sim}^{\S} \leq \Diamond\mathsf{sim}$, and thus $\mathsf{sim}^{\S} = \Diamond\mathsf{sim}$. Consequently, sim is compatible and substitutive (recall that a closed relation is compatible and substitutive if its open extension is).

Lemma B.8 (Key Lemma)**.** *Let* a *be a closed reflexive and transitive relation. Assume*

$$a \leq B(\Diamond a) \qquad \dagger$$

. Then, $a^{\S} \leq \Diamond B(a^{\S})$ *is derivable.*

Proof. For readability, rather than giving a unique big calculation, we split the latter into three parts interleaving some textual comments between them. First, we massage the thesis $a^{\S} \leq \Diamond B(a^{\S})$ so to obtain an equality between closed relations.

$$\begin{aligned}
& a^{\S} \leq \Diamond((b^{\Downarrow} \,;\, \overline{\Diamond a^{\S}}) \leftarrow b^{\Downarrow}) \\
&\iff \square(a^{\S}[\Delta]) \leq (b^{\Downarrow} \,;\, \overline{\Diamond a^{\S}}) \leftarrow b^{\Downarrow} && \text{(K12)} \\
&\iff \square a^{\S} \leq (b^{\Downarrow} \,;\, \overline{\Diamond a^{\S}}) \leftarrow b^{\Downarrow} && \text{(§2) and (§1)} \\
&\iff b^{\Downarrow} \leq \square a^{\S} \rightarrow (b^{\Downarrow} \,;\, \widehat{a^{\S}}) && \text{RA}
\end{aligned}$$

We now proceed by fixed point induction on $b^{\Downarrow}$, whereby we need to prove

$$\square a^{\S} ; (\Delta_\kappa \vee b ; \square a^{\S} \rightarrow (b^{\Downarrow} ; \widehat{a^{\S}}) \vee \langle \square a^{\S} \rightarrow (b^{\Downarrow} ; \widehat{a^{\S}}) \rangle ; b ; \square a^{\S} \rightarrow (b^{\Downarrow} ; \widehat{a^{\S}})) \leq b^{\Downarrow} ; \widehat{a^{\S}}$$

which, by (RA), follows from the three equalities:

$$\Box a^{\S} \,;\, \Delta_\kappa \le b^{\Downarrow} \,;\, \widehat{a^{\S}} \qquad \Theta_1$$

$$\Box a^{\S} \,;\, b \,;\, \Box a^{\S} \to (b^{\Downarrow} \,;\, \widehat{a^{\S}}) \le b^{\Downarrow} \,;\, \widehat{a^{\S}} \qquad \Theta_2$$

$$\Box a^{\S} \,;\, \langle \Box a^{\S} \to (b^{\Downarrow} \,;\, \widehat{a^{\S}}) \rangle \,;\, b \,;\, \Box a^{\S} \to (b^{\Downarrow} \,;\, \widehat{a^{\S}}) \le b^{\Downarrow} \,;\, \widehat{a^{\S}}. \qquad \Theta_3$$

To improve readability, let us abbreviate $\Box a^{\S} \to (b^{\Downarrow} \,;\, \widehat{a^{\S}})$ as c, so to have (by (RA)):

$$\Box a^{\S} \,;\, c \le b^{\Downarrow} \,;\, \widehat{a^{\S}} \qquad \ddagger$$

We now proceed in order. For Θ_1, we calculate:

$$\begin{aligned}
&\Box a^{\S} \,;\, \Delta_\kappa \\
&\le \Box(\Diamond a \,;\, \widehat{a^{\S}}) \,;\, \Delta_\kappa && (\S3)\\
&= \Box\Diamond a \,;\, \Box\widehat{a^{\S}} \,;\, \Delta_\kappa && (\zeta 1)\\
&= \Box\Diamond a \,;\, \Delta_\kappa \,;\, \Box\widehat{a^{\S}} && (\mathrm{RC2})\\
&\le \Box\Diamond a \,;\, b^{\Downarrow} \,;\, \Box\widehat{a^{\S}} && (\Delta_\kappa \le b^{\Downarrow})\\
&= a \,;\, b^{\Downarrow} \,;\, \Box\widehat{a^{\S}} && (\mathrm{K19}) \text{ since } a = \Box a\\
&\le a \,;\, b^{\Downarrow} \,;\, \widehat{a^{\S}} && (\zeta 4)\\
&\le b^{\Downarrow} \,;\, \widehat{\Diamond a} \,;\, \widehat{a^{\S}} && \dagger\\
&= b^{\Downarrow} \,;\, \widehat{\Diamond a \,;\, a^{\S}} && (\mathrm{S25})\\
&\le b^{\Downarrow} \,;\, \widehat{a^{\S}} && (\S4)
\end{aligned}$$

For Θ_2, we calculate:

$$\begin{aligned}
&\Box a^{\S} \,;\, b \,;\, c \\
&\le \Box(\Diamond a \,;\, \widehat{a^{\S}}) \,;\, b \,;\, c && (\S3)\\
&\le \Box\Diamond a \,;\, \Box\widehat{a^{\S}} \,;\, b \,;\, c && (\zeta 1)\\
&= a \,;\, \Box\widehat{a^{\S}} \,;\, b \,;\, c && (\mathrm{K19}) \text{ since } a = \Box a\\
&= a \,;\, \Box\widehat{a^{\S}} \,;\, \Box b \,;\, c && \Box b = b\\
&= a \,;\, \Box(\widehat{a^{\S}} \,;\, b) \,;\, c && (\zeta 1)\\
&\le a \,;\, \Box(\widehat{a^{\S}} \,;\, \langle \Delta_\kappa, \Delta \rangle \,;\, b) \,;\, c && \mathrm{Inv}\\
&\le a \,;\, \Box(\langle a^{\S}, a^{\S} \rangle \,;\, \langle \Delta_\kappa, \Delta \rangle \,;\, b) \,;\, c && (\mathrm{C19})\\
&\le a \,;\, \Box\langle a^{\S} \,;\, \Delta_\kappa, a^{\S} \rangle \,;\, b \,;\, c && (\varphi 5)\\
&= a \,;\, \Box\Box\langle a^{\S} \,;\, \Delta_\kappa, a^{\S} \rangle \,;\, b \,;\, c && (\zeta 5)\\
&\le a \,;\, \Box\langle \Box a^{\S} \,;\, \Box\Delta_\kappa, \Box a^{\S} \rangle \,;\, \Box b \,;\, c && (\varphi 11) \text{ and } (\zeta 1)\\
&= a \,;\, \Box\langle \Box a^{\S} \,;\, \Delta_\kappa, \Box a^{\S} \rangle \,;\, b \,;\, c && \Box\Delta_\kappa = \Delta_\kappa \text{ and } \Box b = b\\
&\le a \,;\, \Box\langle b^{\Downarrow} \,;\, \widehat{a^{\S}}, \Box a^{\S} \rangle \,;\, b \,;\, c && \Theta_1\\
&= a \,;\, \Box\langle b^{\Downarrow}, \Delta \rangle \,;\, \langle \widehat{a^{\S}}, \Box a^{\S} \rangle \,;\, b \,;\, c && (\varphi 5)\\
&\le a \,;\, \Box\langle b^{\Downarrow} \rangle \,;\, \langle \widehat{a^{\S}}, a^{\S} \rangle \,;\, b \,;\, c && (\zeta 4) \text{ and } \langle b^{\Downarrow} \rangle = \langle b^{\Downarrow}, \Delta \rangle\\
&\le a \,;\, \Box\langle b^{\Downarrow} \rangle \,;\, b \,;\, a^{\S}[a^{\S}] \,;\, c && (\text{Harmony}), \text{ since } \widehat{a^{\S}} \le a^{\S}\\
&\le a \,;\, \Box\langle b^{\Downarrow} \rangle \,;\, b \,;\, a^{\S} \,;\, c && (\S2)\\
&\le a \,;\, \Box\langle b^{\Downarrow} \rangle \,;\, \Box b \,;\, \Box a^{\S} \,;\, c && (\zeta 1)\\
&\le a \,;\, \langle b^{\Downarrow} \rangle \,;\, b \,;\, \Box a^{\S} \,;\, c && ?? \text{ and } \Box b^{\Downarrow} = b \text{ and } \Box b = b\\
&\le a \,;\, \langle b^{\Downarrow} \rangle \,;\, b \,;\, b^{\Downarrow} \,;\, \widehat{a^{\S}} && \ddagger\\
&\le a \,;\, b^{\Downarrow} \,;\, \widehat{a^{\S}} && \text{Definition of } b^{\Downarrow}\\
&\le b^{\Downarrow} \,;\, \widehat{\Diamond a} \,;\, \widehat{a^{\S}} && \dagger\\
&\le b^{\Downarrow} \,;\, \widehat{\Diamond a \,;\, a^{\S}} && (\mathrm{S25})\\
&\le b^{\Downarrow} \,;\, \widehat{a^{\S}} && (\S4)
\end{aligned}$$

For Θ_3, we calculate:

$$
\begin{aligned}
&\Box a^{\S} \,;\, \langle c\rangle \,;\, b \,;\, c \\
&\quad \le \Box(\Diamond a \,;\, \widehat{a^{\S}}) \,;\, \langle c\rangle \,;\, b \,;\, c && (\S 3)\\
&\quad = \Box\Diamond a \,;\, \Box\widehat{a^{\S}} \,;\, \langle c\rangle \,;\, b \,;\, c && (\zeta 1)\\
&\quad = a \,;\, \Box\widehat{a^{\S}} \,;\, \langle c\rangle \,;\, b \,;\, c && \text{(K19) since } a = \Box a\\
&\quad = a \,;\, \Box(\widehat{a^{\S}} \,;\, \langle c\rangle) \,;\, b \,;\, c && (\mathrm{RC}5)\\
&\quad = a \,;\, \Box\langle a^{\S}, a^{\S}\rangle \,;\, \langle c, \Delta\rangle \,;\, b \,;\, c && (\mathrm{C}19)\\
&\quad = a \,;\, \Box\langle a^{\S} \,;\, c, a^{\S}\rangle \,;\, b \,;\, c && (\zeta 1)\\
&\quad = a \,;\, \Box\Box\langle a^{\S} \,;\, c, a^{\S}\rangle \,;\, b \,;\, c && (\zeta 5)\\
&\quad = a \,;\, \Box\langle \Box a^{\S} \,;\, \Box c, \Box a^{\S}\rangle \,;\, b \,;\, c && (\zeta 1) \text{ and } (\varphi 11)\\
&\quad = a \,;\, \Box\langle \Box a^{\S} \,;\, c, \Box a^{\S}\rangle \,;\, b \,;\, c && (\zeta 4)\\
&\quad \le a \,;\, \Box\langle b^{\Downarrow} \,;\, \widehat{a^{\S}}, \Box a^{\S}\rangle \,;\, b \,;\, c && \ddagger\\
&\quad \le a \,;\, \Box(\langle b^{\Downarrow}, \Delta\rangle \,;\, \langle \widehat{a^{\S}}, \Box a^{\S}\rangle) \,;\, b \,;\, c && (\mathrm{S}25)\\
&\quad \le a \,;\, \Box(\langle b^{\Downarrow}\rangle \,;\, \langle \widehat{a^{\S}}, \Box a^{\S}\rangle) \,;\, \Box b \,;\, c && \Box b = b \text{ and } \langle b^{\Downarrow}, \Delta\rangle = \langle b^{\Downarrow}\rangle\\
&\quad = a \,;\, \Box(\langle b^{\Downarrow}\rangle \,;\, \langle \widehat{a^{\S}}, \Box a^{\S}\rangle \,;\, b) \,;\, c && (\zeta 1)\\
&\quad \le a \,;\, \Box(\langle b^{\Downarrow}\rangle \,;\, \langle \widehat{a^{\S}}, a^{\S}\rangle \,;\, b) \,;\, c && (\zeta 4)\\
&\quad \le a \,;\, \Box(\langle b^{\Downarrow}\rangle \,;\, b \,;\, a^{\S}[a^{\S}]) \,;\, c && \text{(Harmony), since } \widehat{a^{\S}} \le \widehat{a^{\S}}\\
&\quad \le a \,;\, \Box(\langle b^{\Downarrow}\rangle \,;\, b \,;\, a^{\S}) \,;\, c && (\S 2)\\
&\quad = a \,;\, \Box\langle b^{\Downarrow}\rangle \,;\, \Box b \,;\, \Box a^{\S} \,;\, c && (\zeta 1)\\
&\quad = a \,;\, \langle b^{\Downarrow}\rangle \,;\, b \,;\, \Box a^{\S} \,;\, c && (\varphi 11) \text{ and } \Box b = b \text{ and } \Box b^{\Downarrow} = b\\
&\quad \le a \,;\, \langle b^{\Downarrow}\rangle \,;\, b \,;\, b^{\Downarrow} \,;\, \widehat{a^{\S}} && \ddagger\\
&\quad \le a \,;\, b^{\Downarrow} \,;\, \widehat{a^{\S}} && \text{Definition of } b^{\Downarrow}\\
&\quad \le b^{\Downarrow} \,;\, \widehat{\Diamond a} \,;\, \widehat{a^{\S}} && \dagger\\
&\quad \le b^{\Downarrow} \,;\, \widehat{\Diamond a \,;\, a^{\S}} && (\mathrm{S}25)\\
&\quad \le b^{\Downarrow} \,;\, \widehat{a^{\S}} && (\S 4)
\end{aligned}
$$

□

Theorem B.9. *Open similarity is substitutive and compatible.*

Proof. By Theorem 6.3, $\mathsf{sim}^{\S}$ is an open simulation, hence $\mathsf{sim}^{\S} \le \Diamond\mathsf{sim}$; by coinduction ($\Diamond\mathsf{sim}$ is the greatest solution to $x = \Diamond B(x)$). Since $\Diamond\mathsf{sim} \le \mathsf{sim}^{\S}$, we conclude $\mathsf{sim}^{\S} = \Diamond\mathsf{sim}$ and thus compatibility of substitutivity of $\Diamond\mathsf{sim}$, as sim is □-reflexive and □-transitive and thus $\mathsf{sim}^{\S}$ is compatible and substitutive. □

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009